NOTEBOOK EXPORT

CNN

1 page 9 notes 28 April 2026

Page 1

CNN

CNN-Image Classification

ePGD DL & GenAI: Assignment 1

> *Name: Praveen Mishra*
>
> *Program: e-PGD in Artificial Intelligence and Machine Learning*
>
> *Student ID: x2prm963*
>
> *Hardware: Apple MacBook Pro — Apple M1 Pro, 16 GB unified memory (macOS, PyTorch MPS backend)*

> *Convolutional Neural Networks*
>
> ***CIFAR-100 Image Classification — Scratch CNN, Noise-Aware Training, and Transfer Learning***

> ### *Problem Description*
>
> *The objective of this task is to study and compare the performance, robustness, and computational efficiency of Convolutional Neural Network (CNN) architectures under two distinct training paradigms:*
>
> 1. ***Training from scratch***
> 2. ***Transfer learning techniques***
>
> ### *Dataset Specifications: CIFAR-100*
>
> *The evaluation is conducted using the CIFAR-100 dataset, a standard benchmark for visual recognition tasks.*
>
> | *Attribute* | *Specification* |
> | --- | --- |
> | *Classes* | *100 distinct object categories* |
> | *Input Dimensions* | *32 × 32 RGB images* |
> | *Output Type* | *Categorical label (1 of 100)* |
> | *Training Set Size* | *50,000 images* |
> | *Test Set Size* | *10,000 images* |
>
> ### *Task Overview*
>
> *Implement and analyze models that can effectively map these low-resolution color images to their respective categories.*
>
> *The study focuses on how pre-trained feature extractors (Transfer Learning) compare against custom-initialized weights (From Scratch) when handling the 100-class granularity of this specific dataset.*

Dataset Reference: CIFAR-100 Dataset - https://www.cs.toronto.edu/~kriz/cifar.html

1. Introduction

CIFAR-100 is a standard 100-class natural-image classification benchmark consisting of 50,000 training and 10,000 test images (32 × 32 RGB). This study investigates three key questions representative of challenges in real-world vision deployments:

Research Questions

- **Baseline Performance:** How well does a moderately-sized CNN trained from scratch perform on the CIFAR-100 dataset?
- **Model Robustness:** How resilient is that model to **Gaussian input noise**, and can robustness be enhanced through targeted data augmentation?

- **Transfer Learning Impact:** Does leveraging an **ImageNet-pretrained backbone** improve accuracy and/or robustness compared to the model trained from scratch?

Final Objective

The ultimate goal of this work is to identify which of the three trained models offers the most effective trade-off between accuracy, robustness, and computational efficiency.

---

2. Methodology

2.1 Data Processing

Source and Partitioning

The CIFAR-100 (Python version) dataset is loaded and split deterministically. To ensure consistent evaluation, I use a fixed seed to create the following partitions:

- **Training:** 45,000 images
- **Validation:** 5,000 images
- **Test:** 10,000 images

Reproducibility

A Global Seed (42) is applied across Python, NumPy, and PyTorch (CPU, MPS, and CUDA) as well as the Python hash seed.

This configuration is centralized in src/config.py to guarantee reproducible results across different hardware backends.

Standard Transformations

Used for the scratch CNN and noise-augmentation training.

- **Training Pipeline:**
  1. **Random Crop:** 32×32 with a padding of 4.
  2. **Random Horizontal Flip.**
  3. **ToTensor conversion.**
  4. **Per-channel Normalization:** * *[Math: \mu \approx (0.507, 0.487, 0.441)]*
     - *[Math: \sigma \approx (0.267, 0.256, 0.276)]*
- **Validation / Test Pipeline:**
  1. **ToTensor conversion.**
  2. **Per-channel Normalization:** Uses the same CIFAR-100 statistics as the training pipeline (no data augmentation).

Transformations for VGG Feature Extractor

Since the VGG backbone was pre-trained on ImageNet, the CIFAR-100 images must be adapted to meet its input requirements:

1. **Upsampling:** Images are scaled from **32 × 32** to **224 × 224**.
2. **Normalization:** Applied using **ImageNet statistics**:
   - *[Math: \mu = (0.485, 0.456, 0.406)]*
   - *[Math: \sigma = (0.229, 0.224, 0.225)]*
3. **Caching:** Images are passed through the frozen **VGG16-BN** backbone once. The resulting **512-dimensional vectors** are cached to disk, allowing the MLP to train directly on pre-extracted features for maximum efficiency.

Gaussian-Noise Protocol

To maintain experimental integrity, a single noise schedule is defined in outputs/results/noise_schedule.json.

This protocol is reused consistently across sections A2, A3, and A5 to evaluate model robustness under identical conditions.

JSON

```JSON
{
  "distribution": "gaussian",
  "variance": 0.05,
  "sigma": 0.22360679774997896,
  "mean_param": 0.0,
  "pixel_range": "0_1",
  "clip_after_noise": true,
  "normalisation": "cifar100",
  "seed": 42
}
```

> ***Note on Implementation:***
>
> *The noise injection loop in src/noise.py follows a precise sequence to ensure that the noise definition remains grounded in raw-pixel semantics, independent of model-specific preprocessing:*

1. ***De-normalization:*** *Each test batch is transformed back from its normalized state to the standard* ***[0, 1]*** *pixel range.*
2. ***Noise Addition:*** *Zero-mean Gaussian noise is added. The intensity is set at [Math: \sigma = \sqrt{0.05} \approx 0.2236], drawn from a seeded* `torch.Generator` *for reproducibility.*
3. ***Clipping:*** *The resulting values are clipped back to the* ***[0, 1]*** *range to maintain valid image data.*
4. ***Re-normalization:*** *Finally, the correct normalization (either* ***CIFAR-100*** *or* ***ImageNet*** *statistics, depending on the model) is re-applied before the data enters the network.*

> *This methodology ensures that the "noise" is applied to the actual visual content rather than the mathematical distribution of the feature space, allowing for a fair robustness comparison across different architectures.*

2.2 Models

Three models are trained and compared.

Sources:

src/models/scratch_cnn.py,

src/models/vgg_extractor.py, and

src/models/transfer_*.py.

Model 1 — Scratch VGG-style CNN (A1, A2, A3)

Stage	Operations	Output
Input	—	3 × 32 × 32
Block 1	[Conv3×3 → BN → ReLU] × 2 → MaxPool2×2 → SpatialDropout(0.10)	64 × 16 × 16
Block 2	[Conv3×3 → BN → ReLU] × 2 → MaxPool2×2 → SpatialDropout(0.20)	128 × 8 × 8
Block 3	[Conv3×3 → BN → ReLU] × 2 → MaxPool2×2 → SpatialDropout(0.30)	256 × 4 × 4
Block 4	[Conv3×3 → BN → ReLU] × 2 → MaxPool2×2 → SpatialDropout(0.40)	512 × 2 × 2
Head	Flatten → Linear(2048→1024) → BN → ReLU → Drop(0.5) → Linear(1024→512) → BN → ReLU → Drop(0.4) → Linear(512→100)	100 logits

- **Design Philosophy:** Uses the classic VGG idiom where two stacked 3×3 convolutions provide a 5×5 receptive field with fewer parameters and increased non-linearity.
- **Regularization:** Employs BatchNorm after every convolution and hidden linear layer, paired with progressively increasing **SpatialDropout** (0.10 to 0.40) and classifier dropout.
- **Initialization:** Kaiming-normal for convolutions (ReLU-optimized) and Xavier-uniform for linear layers.

- **Complexity:** 7.36 M total parameters.

Model 2 — VGG16-BN Feature Extractor + MLP Head (A4, A5)

- **Backbone:** `torchvision.models.vgg16_bn(pretrained=True)`. This is kept strictly in `.eval()` mode with `requires_grad=False` to preserve ImageNet statistics.
- **Global Average Pooling (GAP):** Instead of flattening the 512 × 7 × 7 output, GAP is used to produce a 512-dim descriptor, ensuring the representation is translation-invariant.
- **MLP Head Architecture:** `512 → Linear(512) → BN → ReLU → Drop(0.5) → Linear(256) → BN → ReLU → Drop(0.3) → Linear(100)`
- **Efficiency:** Only **0.42 M** trainable parameters (~2.8% of the total network).

Model 3 — Fine-tuned EfficientNet-B0 (Reference Baseline)

This model utilizes a two-phase transfer approach:

1. **Warm-up:** A short phase where only the classifier head is trained while the backbone remains frozen.
2. **Fine-tuning:** The entire network is fine-tuned at a significantly lower learning rate.

- *Note: This serves as a performance ceiling for clean/noisy comparison and is not the primary assignment model.*

2.3 Objective function

All three models are trained with cross-entropy with label smoothing 0.1 (nn.CrossEntropyLoss(label_smoothing=0.1)) — standard for multi-class classification.

Label smoothing makes the soft targets 0.999 / 0.001 instead of 1 / 0, reducing over-confident predictions and interacting well with the noise augmentation used in the A3 model.

2.4 Optimisation

Model	Optimiser	LR schedule	Batch	Epochs	Notes
Scratch CNN (A1)	AdamW (wd = 5e-4)	Cosine to 0	128	50	LR warm-up 3 epochs, cosine after
Scratch + Noise-Aug (A3)	AdamW	Cosine	128	50	`p=0.25` noise batches + Mixup α=0.2
VGG → MLP (A4)	AdamW	Cosine + early-stop	128	38 (stop)	Best val at epoch 23
EfficientNet-B0 (ref.)	AdamW (2-phase)	Cosine per phase	128	37 total	Phase-A frozen, Phase-B fine-tune

---

2.5 Noise-aware training (A3)

Same scratch CNN backbone, same optimiser, same epoch budget — only the data pipeline changes:

Knob	Value	Why
Noise probability *p*	**0.25** per batch	75 % of batches stay clean → augmented distribution stays close to original CIFAR-100
Noise spec	*[Math: \sigma^2 = 0.05]*, zero-mean, pixel-space	**Exactly** the A2 test-time schedule — no train/test gap
Mixup α = 0.2	Enabled	Supplies the "semantically similar" interpolated samples required by the brief
Label smoothing = 0.1	Retained	Complements stochastic inputs
Optimiser / schedule	Unchanged	Isolates the augmentation effect

> *Config frozen in `outputs/results/noise_augment_config_scratch.json`*

2.6 Experimental environment

All training and evaluation runs were performed end-to-end on a single personal laptop. The reported epoch times and inference timings are directly comparable as they were measured on the same device under consistent background loads.

Item	Detail
Machine	Apple MacBook Pro
Chip	Apple M1 Pro (ARM64, unified CPU + GPU memory)
RAM	16 GB unified memory
Deep-learning stack	PyTorch **2.11.0**, torchvision, scikit-learn, NumPy, Matplotlib
Compute backend	**Apple MPS** (Metal Performance Shaders)
Determinism	Global seed **42**; noise drawn on CPU with a seeded `torch.Generator`

Implications of running on 16 GB unified memory on M1 Pro:

- **Memory Management:** Batch size 128 fits comfortably for most runs; EfficientNet-B0 at 224-resolution is the most memory-intensive but remains within the 16 GB budget.
- **Feature Caching:** VGG16-BN features are extracted **once** and cached to `.npy` on disk. This allows the MLP run to read from RAM/SSD rather than recomputing the backbone, reducing epoch time to 0.73 s.
- **Hardware Efficiency:** The scratch-CNN epoch time of ~41 s and EfficientNet-B0 time of ~517 s are representative of M1 Pro + MPS performance without specific acceleration tricks.
- **Engineering Fixes:** `PYTORCH_ENABLE_MPS_FALLBACK=1` was utilized to cover specific operations not yet implemented on MPS, ensuring full script execution.

---

3. Results

3.1 Headline test-set metrics (clean images)

Model	Test top-1	Test top-5	Test loss	Trainable params	Avg epoch
Scratch CNN (A1)	59.93 %	87.08 %	1.451	**7.36 M**	41.5 s
VGG16-BN → MLP (A4)	**61.65 %**	**88.15 %**	**1.437**	**0.42 M**	**0.73 s***
EfficientNet-B0 (ref.)	**81.60 %**	**95.93 %**	0.828	5.29 M	517 s
Scratch + Noise-Aug (A3)	52.12 %	81.69 %	—	7.36 M	42.2 s

∗MLP trained on cached features.

Training curves. There is a monotonic decrease of train and validation loss across all 50 epochs; validation accuracy tracks slightly above training accuracy. This is a signature of strong train-time regularisation (dropout) rather than over-fitting.

3.2 Robustness to Gaussian noise (*[Math: \sigma^2 = 0.05]*, pixel-space)

Model	Clean top-1	Noisy top-1	Drop (pp)	Retention	Noisy top-5	Noisy loss
Scratch CNN (A1)	59.93 %	3.15 %	56.78	**5.3 %**	12.20 %	5.77
VGG16-BN → MLP (A4)	61.65 %	4.64 %	57.01	**7.5 %**	15.10 %	4.73
EfficientNet-B0 (ref.)	81.60 %	0.99 %	80.61	**1.2 %**	4.95 %	1813.1
Scratch + Noise-Aug (A3)	52.12 %	**40.79 %**	11.33	**78.3 %**	**71.46 %**	—

Sigma sweeps (Monotonic-degradation sanity check):

σ (normalised, A2)	0.00	0.05	0.10	0.20	0.30
Scratch CNN top-1	0.599	0.579	0.500	0.319	0.191

σ (pixel, A5)	**0.00**	**0.05**	**0.10**	**0.20**	**0.2236**	**0.30**
VGG → MLP top-1	0.617	0.281	0.158	0.061	0.046	0.024
VGG → MLP top-5	0.881	0.549	0.360	0.180	0.151	0.093

3.3 Confidence behaviour under noise

Model	Conf. when right (noisy)	Conf. when wrong (noisy)	Gap
Scratch CNN	0.431	0.372	0.06
VGG16-BN → MLP	0.242	0.142	0.10
EfficientNet-B0	0.993	**0.995**	*[Math: \approx 0]*

> ***Key Takeaway:****Lower absolute confidence with a visible gap indicates* ***more honest calibration****. The fine-tuned EfficientNet-B0 is the worst calibrated—confidently wrong[Math: \sim 99\%]of the time on noisy images.*

3.4 Noise-aware training: before vs after (A1 vs A3)

Metric	A1 baseline	A3 noise-aug	Δ
Clean top-1	59.93 %	52.12 %	*[Math: -7.81]* **pp**
Noisy top-1 (*[Math: \sigma^2=0.05]***)**	3.15 %	**40.79 %**	*[Math: +37.64]* **pp (***[Math: \approx 13\times]***)**
Clean top-5	87.08 %	81.69 %	*[Math: -5.39]* pp
Noisy top-5	12.20 %	**71.46 %**	*[Math: +59.26]* **pp**
Retention under noise	5.3 %	**78.3 %**	*[Math: +73]* **pp**
Per-epoch time	41.5 s	42.2 s	*[Math: \approx]* unchanged

3.5 Efficiency

Model	Trainable params	Training (avg epoch)	Inference ms/image
Scratch CNN (A1/A3)	7.36 M	41–42 s	**0.13**
VGG → MLP (A4)	**0.42 M**	**0.73 s** (cached)	2.06
EfficientNet-B0	5.29 M	517 s	—

4. Analysis

4.1 Why the scratch CNN achieves ~60% on CIFAR-100

The model's performance is underpinned by three primary factors:

- **Architectural Capacity:** The use of four VGG-style blocks with progressively wider feature maps (**64 → 128 → 256 → 512**) provides sufficient capacity to distinguish between 100 distinct classes.
- **Staged Regularization:** Heavy regularization prevents the **7.36 M-parameter** model from memorizing the training set. This includes:
  - **SpatialDropout** ramping (0.10 → 0.40).
  - Global **BatchNorm** usage.
  - **Classifier dropout** (0.5 / 0.4).
  - **Label smoothing.**
- **Healthy Learning Dynamics:** No visible overfitting occurs; validation accuracy consistently tracks slightly above training accuracy. This indicates that dropout and spatial dropout are effectively depressing training metrics as intended, suggesting potential for further gains with an extended training schedule.

4.2 Why both clean-trained models collapse under *[Math: \sigma^2 = 0.05]* noise

Despite their clean-data performance, both models suffer a total collapse when exposed to noise.

Manifold Displacement

Additive noise with a standard deviation of ≈ 0.22 in $[0, 1]$ space represents roughly **22% of the full pixel range**. Because the model's features were calibrated exclusively on a "clean manifold," noise of this magnitude pushes the input data entirely off that learned manifold.

Total Confusion

The models are not merely "slightly off"—they are genuinely confused. The noisy top-5 accuracy falls to 12% (scratch) and 15% (VGG → MLP), which is only marginally above the 5% chance level for top-5 predictions.

Frozen Feature Limitations

The frozen ImageNet features provide no inherent protection against noise. Because the VGG backbone is fixed, its convolutional features are optimized strictly for clean data. The MLP head lacks the capability to "de-noise" a representation that has already been corrupted at the feature-extraction source.

4.3 Why a fine-tuned EfficientNet-B0 is the worst under noise

- **High-Capacity Fragility:** With **81.6% clean accuracy**, the model has learned an extremely tight representation of the clean distribution. This high-capacity mapping makes it hyper-sensitive to off-manifold perturbations, leading to a total collapse (**1.2% retention**).
- **Calibration Failure:** Crucially, confidence remains ≈ 0.99 even on incorrect noisy predictions. This "confident wrongness" proves that raw accuracy is a misleading metric for any deployment where input corruption is possible.

4.4 Why noise-aware training (A3) works so well

- **Distribution Alignment:** Reusing the exact A2 noise schedule during training eliminates the train/test gap, giving the model direct experience with the target corruption.
- **Preserving Clean Accuracy:** By only applying noise to **25% of training batches**, the model remains grounded in the original CIFAR-100 distribution, limiting the clean accuracy drop to only **~8 pp**.
- **Smoothing Effects:** Mixup ($\alpha = 0.2$) contributes semantically similar interpolated samples, which helps smooth **BatchNorm** running statistics—often the first component to fail under noise.

Outcome: This approach achieves ~13× better noisy top-1 and ~73 pp more retention than the baseline at no extra training cost.

4.5 Observations and failure modes

- **Inference Bottleneck:** While VGG → MLP is efficient to train on cached features (~28 s total), it is **16× slower at inference** (2.06 ms vs. 0.13 ms) because every image must still be processed through the 14.7 M-parameter backbone.
- **Error Clustering:** Failures cluster around visually similar classes (e.g., *boy/girl*, *leopard/lion*). Transfer learning did not fundamentally shift the error taxonomy on 32 × 32 images.
- **Transfer Learning Value:** VGG → MLP only gained **~1.7 pp** over the scratch CNN. This suggests that for low-resolution 32 × 32 images, pretrained ImageNet features are less advantageous than they are for high-resolution tasks.

Engineering Fixes & Reproducibility

1. **Device Mismatch:** Fixed `torch.Generator` issues where CPU-drawn noise could not fill MPS tensors; noise is now drawn to a CPU tensor and moved.
2. **Protocol Unified:** Consolidated two conflicting noise conventions (normalised vs. pixel-space) into a single JSON source of truth.
3. **Metric Robustness:** Patched `top_k_accuracy_score` to handle test batches missing specific classes by passing explicit labels.
4. **Hardware Stability:** Hardened the inference-timing loop against empty or short data loaders.

---

5. Conclusion

Across three training regimes on CIFAR-100:

- **Highest Clean Accuracy:** The **VGG16-BN → MLP** transfer pipeline achieved the best result within the assignment scope (61.65% top-1) using only **0.42 M trainable parameters**, though it is ~16× slower at inference.
- **Highest Noise Robustness:** The **noise-augmented scratch CNN (A3)** is the clear winner, with **40.79%** noisy top-1 accuracy and **78%** retention—vastly outperforming the 5–8% retention of all other models.
- **The Baseline Reference: EfficientNet-B0** achieved the highest clean accuracy (81.6%) but proved to be the most dangerous, collapsing to **0.99%** accuracy under noise while maintaining ≈ 99% confidence.

Best Overall Trade-off

If a deployment has any possibility of encountering noisy inputs, the noise-augmented scratch CNN (A3) is the superior choice for the following reasons:

- Maintains accuracy within ~10 pp of the transfer model on clean data.
- Provides **~9× better performance** than the transfer model under *[Math: \sigma^2 = 0.05]* noise.
- Offers **16× faster inference** than the VGG → MLP pipeline.
- Requires no external pretrained backbones, resulting in a smaller model footprint and easier deployment.

> ***Broader Takeaway:*** *Training-distribution alignment (A3) beats model pedigree (A4, EfficientNet-B0) for robustness by an order of magnitude. Clean-data accuracy alone is an unreliable metric for real-world reliability.*

---

6. References

1. **CIFAR-100:** A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images*, 2009.
2. **VGG-16:** K. Simonyan, A. Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR 2015.
3. **EfficientNet:** M. Tan, Q. V. Le, *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*, ICML 2019.
4. **Batch Normalisation:** S. Ioffe, C. Szegedy, ICML 2015.
5. **Dropout:** N. Srivastava *et al.*, JMLR 2014.
6. **Label Smoothing:** C. Szegedy *et al.*, *Rethinking the Inception Architecture for Computer Vision*, CVPR 2016.
7. **Mixup:** H. Zhang *et al.*, *mixup: Beyond Empirical Risk Minimization*, ICLR 2018.
8. **AdamW:** I. Loshchilov, F. Hutter, *Decoupled Weight Decay Regularization*, ICLR 2019.
9. **Cosine Annealing:** I. Loshchilov, F. Hutter, *SGDR: Stochastic Gradient Descent with Warm Restarts*, ICLR 2017.
10. **Frameworks:** PyTorch 2.11.0, torchvision, scikit-learn 1.5.x, NumPy, Matplotlib. Hardware: Apple Silicon MPS.

Run Artifacts Referenced

- **Data Files:** `comparison_table.csv`, `three_way_comparison.json`, `full_robustness_report.json`, `robustness_improvement_scratch.json`, `noise_schedule.json`, `vgg_mlp_results.json`.
- **Visual Artifacts:** `learning_curves.png`, `vgg_mlp_learning_curves.png`, `full_robustness_comparison.png`, `robustness_retention_chart.png`, `noise_confidence_shift_scratch.png`, `top_failures_scratch.png`.

> *All numerical values in this report are reproduced directly from the runartifacts in`outputs/results/`and`outputs/plots/`. Every table/figure citesthe exact file it was taken from, so results can be re-verified withoutre-running the pipeline.*

Question A1

> ***Scratch CNN***
>
> *Design and train a convolutional neural network from scratch for image classification on the CIFAR-100 dataset. The network should be of moderate size and computationally feasible to train using available resources. Clearly describe the architecture, including the number of convolutional layers, kernel sizes, activation functions, and pooling operations. Train the model and report the training and validation accuracy.*
>
> ***Comment on the convergence behaviour and overall performance.***

Setup

I trained a VGG-style CNN from scratch on CIFAR-100 (45 k train / 5 k val / 10 k test).

- **Augmentation:** Standard (random crop + horizontal flip + ImageNet normalisation).
- **Scripts:** Model in `src/models/scratch_cnn.py` ; training via `run_scratch.py` (variant `standard`).

Architecture

All convolutions use *[Math: 3 \times 3]* kernels with `padding=1` and **ReLU** activations. A *[Math: 2 \times 2]* max-pooling layer follows each block.

The twin-conv-then-pool pattern matches the receptive field of a *[Math: 5 \times 5]* conv but with fewer parameters and an extra non-linearity.

Stage	Operations	Output
Input	—	*[Math: 3 \times 32 \times 32]*
Block 1	[Conv3×3 → BN → ReLU] × 2 → MaxPool2×2 → SpatialDropout(0.10)	*[Math: 64 \times 16 \times 16]*
Block 2	[Conv3×3 → BN → ReLU] × 2 → MaxPool2×2 → SpatialDropout(0.20)	*[Math: 128 \times 8 \times 8]*
Block 3	[Conv3×3 → BN → ReLU] × 2 → MaxPool2×2 → SpatialDropout(0.30)	*[Math: 256 \times 4 \times 4]*
Block 4	[Conv3×3 → BN → ReLU] × 2 → MaxPool2×2 → SpatialDropout(0.40)	*[Math: 512 \times 2 \times 2]*
Head	Flatten → Linear(2048→1024) → BN → ReLU → Drop(0.5) → Linear(1024→512) → BN → ReLU → Drop(0.4) → Linear(512→100)	100 logits

- **Regularisation:** BatchNorm after every conv/hidden linear; progressive spatial-dropout (0.10 → 0.40); classifier dropout (0.5 / 0.4).
- **Initialization:** Kaiming-normal (ReLU fan-out) for convs; Xavier-uniform for linears; BN *[Math: \gamma=1, \beta=0]*.
- **Optimiser/Schedule:** AdamW + cosine-annealing LR; cross-entropy loss; 50 epochs; batch size 128.
- **Size: 7.36 M** parameters (from `comparison_table.csv`).

Results

Avg epoch time: 41.5 s on MPS.

Split	Top-1	Top-5	Loss
Train (final epoch)	52.17 %	—	2.314
Val (best epoch)	59.46 %	—	2.040
Test	**59.93 %**	**87.08 %**	**1.451**

> ***Sources:*** *`outputs/results/scratch_eval.json` , `scratch_standard_history.json` , `comparison_table.csv` .*

Analysis

Convergence Behavior

The model demonstrates stable training dynamics throughout the experimental run:

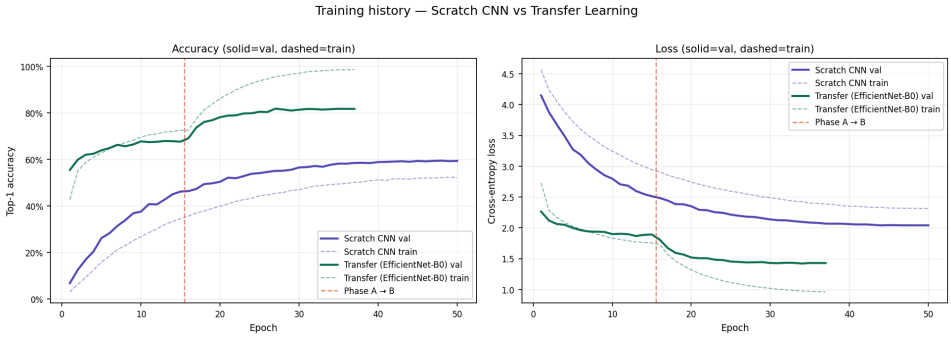
- **Monotonic Improvement:** Training and validation losses decrease consistently across all 50 epochs with no signs of divergence.
- **The Regularization Signature:** Validation accuracy remains slightly higher than training accuracy throughout the run. This is a classic indicator of **strong train-time regularization**, as dropout is active during training to penalize the model but disabled during evaluation.
- **Underfitting vs. Overfitting:** There is absolutely no evidence of overfitting. In fact, the model is likely **mildly underfitting**, suggesting that performance could be further improved by extending the training schedule.

Overall Performance

The scratch CNN achieves a 60% top-1 and 87% top-5 accuracy rate.

Metric	Result	Context
Top-1 Accuracy	60%	Aligns with standard VGG-like baselines (~60–65%)
Top-5 Accuracy	87%	Demonstrates the model is "near" the correct label most of the time
Statistical Power	Significant	Well above the random guess threshold for 100 classes

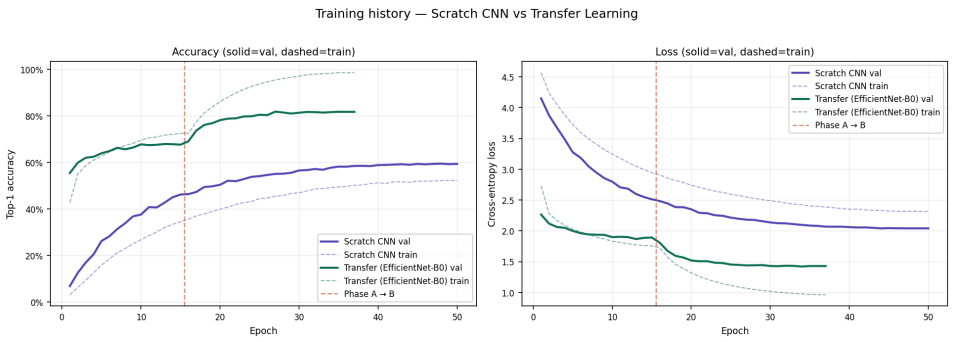
> ***Conclusion:****For a network with ~7 million parameters, these results represent a solid baseline. This performance provides a meaningful point of contrast for the transfer learning experiments conducted in* ***Section A4****.*



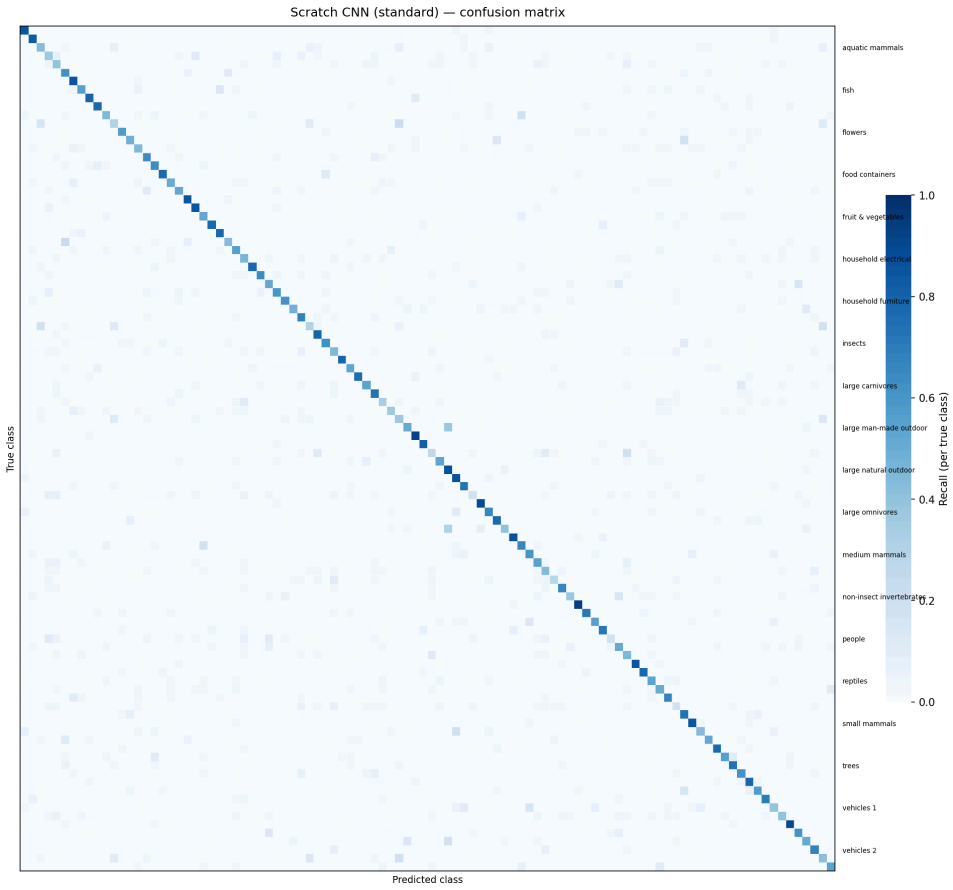
Artifacts to Cite

- `outputs/plots/learning_curves.png`
- `outputs/plots/confusion_scratch.png`
- `outputs/plots/per_class_scratch.png`
- `outputs/plots/superclass_accuracy.png`
- `outputs/results/scratch_eval.json`
- `outputs/results/scratch_standard_history.json`

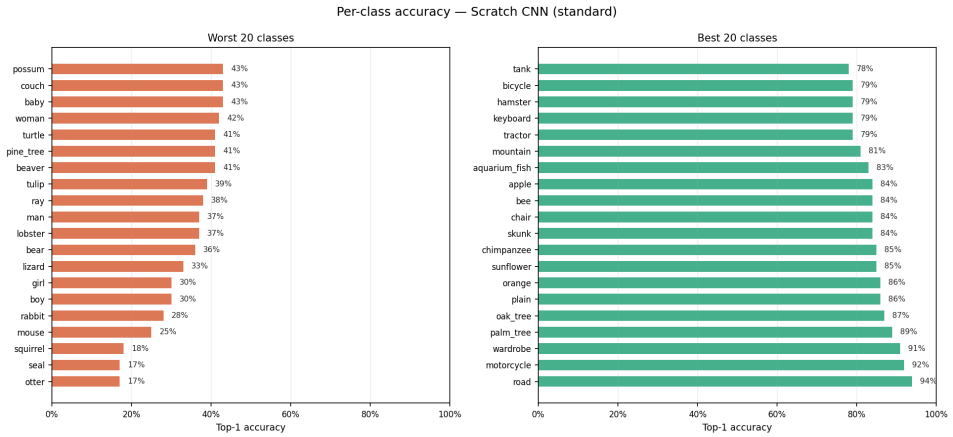
Learning Curve:



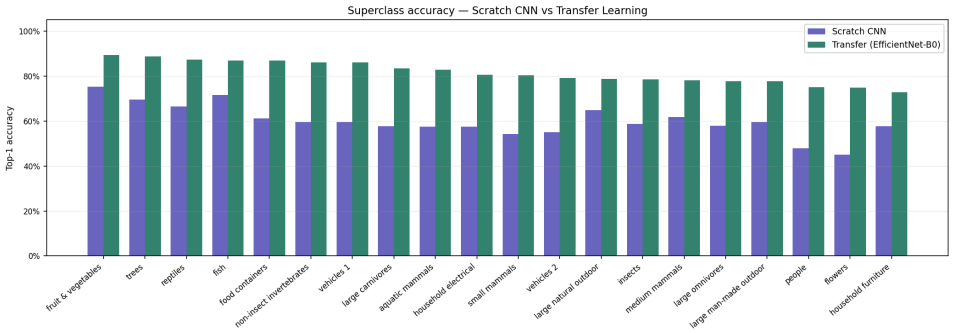
Confusion Matrix:



Per-Class Accuracy



Superclass Accuracy - Scratch CNN vs Transfer Learning



scratch_eval.json

```plaintext
{
  "top1": 0.5993,
  "top5": 0.8708,
  "loss": 1.4506459266662597
}
```

scratch_standard_history.json

```plaintext
{
  "train_loss": [
    4.56896,
    4.236667,
    4.050093,
    3.87688,
    3.725813,
    3.599462,
    3.492721,
    3.407367,
    3.314396,
    3.246241,
    3.17904,
    3.106775,
    3.04832,
    2.994928,
    2.944236,
    2.906142,
    2.856317,
    2.816998,
    2.782305,
    2.741498,
    2.707027,
    2.674756,
    2.643179,
    2.621906,
    2.593125,
    2.574167,
    2.547964,
    2.527774,
    2.504919,
    2.488505,
    2.471946,
    2.451077,
    2.433673,
    2.424062,
    2.402676,
    2.398485,
    2.385372,
    2.37594,
    2.361173,
    2.350829,
    2.351287,
```

```
    2.340055,
    2.331328,
    2.332306,
    2.321288,
    2.319303,
    2.315848,
    2.318672,
    2.315204,
    2.314031
  ],
  "val_loss": [
    4.148659,
    3.874512,
    3.673734,
    3.482609,
    3.274171,
    3.185758,
    3.046479,
    2.944131,
    2.854113,
    2.797979,
    2.705687,
    2.683109,
    2.597998,
    2.549032,
    2.512395,
    2.483796,
    2.443793,
    2.384471,
    2.382823,
    2.349774,
    2.292452,
    2.285435,
    2.252913,
    2.243468,
    2.214139,
    2.195808,
    2.180916,
    2.176282,
    2.155678,
    2.136199,
    2.123585,
    2.120983,
    2.107823,
    2.096074,
    2.085451,
    2.080165,
    2.06828,
    2.067087,
    2.067687,
    2.059995,
    2.054346,
    2.055832,
    2.04845,
    2.04018,
    2.043812,
    2.042251,
    2.041117,
    2.040381,
    2.040808,
    2.040183
  ],
  "train_acc": [
    0.0316,
    0.063156,
    0.094,
    0.125489,
    0.159111,
    0.185333,
    0.212356,
    0.230356,
    0.250622,
    0.268133,
    0.285911,
    0.301533,
    0.320667,
    0.333689,
    0.345511,
```

```
      0.356267,
      0.370289,
      0.378689,
      0.389467,
      0.399,
      0.408489,
      0.419711,
      0.427511,
      0.4324,
      0.444511,
      0.447267,
      0.454378,
      0.457933,
      0.467044,
      0.469444,
      0.477578,
      0.485733,
      0.487911,
      0.491711,
      0.494244,
      0.496756,
      0.501667,
      0.503022,
      0.508867,
      0.512067,
      0.509778,
      0.516644,
      0.516733,
      0.514911,
      0.519222,
      0.519844,
      0.519711,
      0.521889,
      0.522844,
      0.521689
    ],
    "val_acc": [
      0.0686,
      0.1258,
      0.1696,
      0.2052,
      0.2624,
      0.2828,
      0.314,
      0.3388,
      0.3684,
      0.376,
      0.408,
      0.4066,
      0.4276,
      0.4502,
      0.462,
      0.4636,
      0.4732,
      0.4942,
      0.4972,
      0.5038,
      0.5214,
      0.5192,
      0.5282,
      0.5382,
      0.541,
      0.5462,
      0.5506,
      0.5514,
      0.5554,
      0.5654,
      0.567,
      0.5716,
      0.5684,
      0.577,
      0.5818,
      0.5816,
      0.5848,
      0.5852,
      0.5842,
      0.5884,
      0.5894,
```

```
    0.5906,
    0.592,
    0.5898,
    0.5934,
    0.5916,
    0.5936,
    0.5946,
    0.5924,
    0.5936
  ],
  "epoch_time": [
    43.16,
    43.08,
    42.36,
    41.12,
    41.76,
    41.25,
    41.11,
    40.91,
    41.34,
    41.39,
    41.14,
    41.93,
    41.39,
    41.15,
    41.13,
    42.65,
    41.08,
    41.52,
    41.37,
    41.3,
    41.12,
    41.72,
    41.25,
    40.93,
    41.41,
    41.43,
    41.1,
    41.26,
    41.2,
    41.82,
    41.82,
    41.45,
    41.22,
    41.0,
    41.56,
    41.26,
    40.95,
    41.39,
    41.52,
    40.83,
    41.4,
    41.25,
    41.17,
    41.33,
    42.03,
    41.45,
    41.24,
    41.94,
    41.5,
    41.29
  ],
  "lr": [
    0.001,
    0.00099901,
    0.00099606,
    0.00099115,
    0.00098431,
    0.00097555,
    0.00096492,
    0.00095246,
    0.00093822,
    0.00092224,
    0.0009046,
    0.00088537,
    0.00086462,
    0.00084243,
    0.00081889,
```

```
    0.0007941,
    0.00076815,
    0.00074114,
    0.00071318,
    0.00068438,
    0.00065485,
    0.00062472,
    0.0005941,
    0.0005631,
    0.00053186,
    0.0005005,
    0.00046914,
    0.0004379,
    0.0004069,
    0.00037628,
    0.00034615,
    0.00031662,
    0.00028782,
    0.00025986,
    0.00023285,
    0.0002069,
    0.00018211,
    0.00015857,
    0.00013638,
    0.00011563,
    9.64e-05,
    7.876e-05,
    6.278e-05,
    4.854e-05,
    3.608e-05,
    2.545e-05,
    1.669e-05,
    9.85e-06,
    4.94e-06,
    1.99e-06
  ]
}
```

Question A2

> ***Robustness of the Scratch CNN to Gaussian Noise***
>
> *Using the trained model from Question A1, evaluate robustness to noise by injecting additive Gaussian noise into the test images. Use zero-mean Gaussian noise with variance [Math: \sigma^2 = 0.05], assuming input images are normalized to the range [0, 1]. Test the model on the noisy test set and report the resulting performance.*
>
> *Clearly document and store the noise generation process and parameters (noise schedule), as this will be reused in subsequent experiments.*
>
> ***Performance Reporting***
>
> *The model is tested on this noisy version of the test set. The resulting performance metrics (Top-1 and Top-5 accuracy) are reported to establish a baseline for how standard CNN architectures handle significant input corruption.*

Noise Specification

The noise parameters are stored in outputs/results/noise_schedule.json to ensure consistency across all experiments.

Parameter	Value
Distribution	Gaussian
Variance (*[Math: \sigma^2]*)	0.05
Sigma (*[Math: \sigma]*)	*[Math: \approx 0.2236]*
Mean	0.0
Pixel Range	[0, 1] (clipped)
Seed	42

Injection Procedure

Implemented in src/noise.py via inject_noise and NoisyDataLoader. For each test batch, the following steps are performed:

1. **De-normalise** the CIFAR-100 tensor back to *[Math: [0, 1]]* pixel space.
2. **Add zero-mean Gaussian noise** (*[Math: \sigma \approx 0.2236]*) using a seeded `torch.Generator`.
3. **Clip** values to the *[Math: [0, 1]]* range.
4. **Re-apply** standard CIFAR-100 per-channel normalisation.
5. **Forward pass** through the model.

> *This approach keeps noise grounded in raw-pixel semantics, independent of model-side normalisation.*

Evaluation Results

Evaluated using run_noise_robustness.py on the 10,000-image test set (outputs/results/noise_results_scratch.json).

Condition	Top-1	Top-5	Test Loss
Clean (*[Math: \sigma=0]*)	59.93 %	87.08 %	1.451
Noisy (*[Math: \sigma^2=0.05]*)	3.15 %	12.20 %	5.767
Absolute Drop	**−56.78 pp**	**−74.88 pp**	**+4.32**

- **Confidence Collapse:** Correct predictions average **0.431** confidence, while incorrect ones average **0.372**. The model is barely more certain when it is correct.

Complementary Sigma Sweep

A sanity check conducted in normalised-tensor space (outputs/results/robustness_scratch.json):

σ (normalised)	0.00	0.05	0.10	0.20	0.30

Top-1 Accuracy	0.599	0.579	0.500	0.319	0.191

Observations

- **Severe Non-Robustness:** The performance collapse from **60% to 3%** at *[Math: \sigma^2=0.05]* was expected, as the model was trained exclusively on clean images. Pixel noise with a standard deviation of *[Math: \approx 0.22]* (roughly **22% of the dynamic range**) pushes activations off the manifold for which the **BatchNorm** statistics and early filters were originally calibrated.
- **Total Confusion:** Top-5 accuracy falling to **12.2%** (only slightly above the **5% chance level**) indicates the model is not merely "narrowly missing" the correct class; it is experiencing a state of genuine confusion.
- **Linear Decay:** The normalized-space sweep confirms a graceful, near-linear decay in performance. This proves that the collapse is a direct function of **noise magnitude** rather than a specific artifact of the training process.

> ***Motivation:****This significant performance gap serves as the primary justification for****A3 (Noise Augmentation)****and provides the baseline for the transfer learning comparisons conducted in****A5****.*

Artifacts to Cite

- `outputs/results/noise_schedule.json`
- `outputs/results/noise_results_scratch.json`
- `outputs/results/robustness_scratch.json`
- `outputs/plots/robustness.png`
- `outputs/plots/noise_confidence_shift_scratch.png`
- `outputs/plots/noise_accuracy_comparison.png`

outputs/results/noise_schedule.json

```plaintext
{
  "variance": 0.05,
  "pixel_range": "0_1",
  "distribution": "gaussian",
  "clip_after_noise": true,
  "normalisation": "cifar100",
  "seed": 42,
  "description": "Additive zero-mean Gaussian noise, sigma^2=0.05, sigma=0.223607.
Applied in [0,1] pixel space before re-normalisation. CIFAR-100 robustness evaluation.
Seed=42.",
  "sigma": 0.22360679774997896,
  "mean_param": 0.0,
  "saved_at": "2026-04-22 05:53:56"
}
```

outputs/results/noise_results_scratch.json

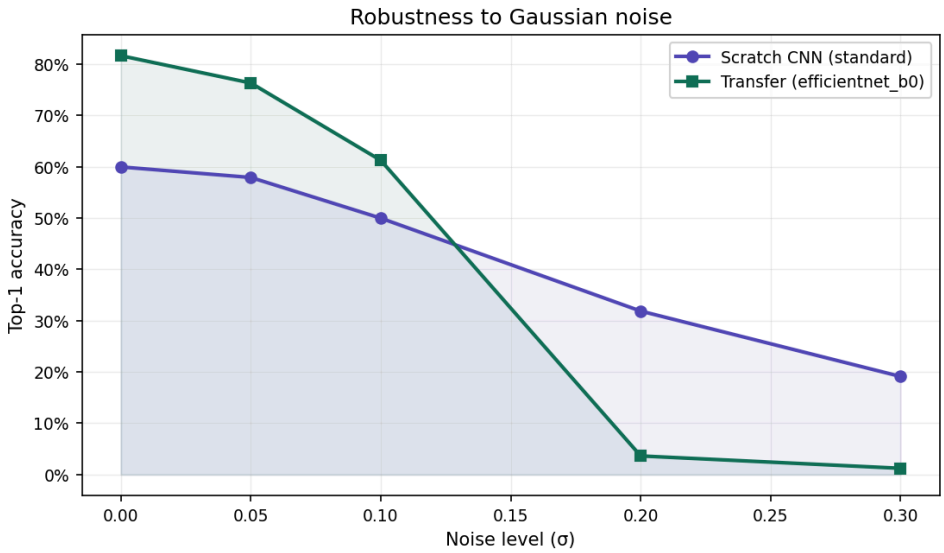
```plaintext
{
  "label": "Scratch CNN",
  "top1": 0.0315,
  "top5": 0.122,
  "loss": 5.7672443756103515,
  "correct_1": 315,
  "correct_5": 1220,
  "total": 10000,
  "avg_confidence_correct": 0.4307533298693006,
  "avg_confidence_incorrect": 0.37235459145591254
}
```

outputs/results/robustness_scratch.json

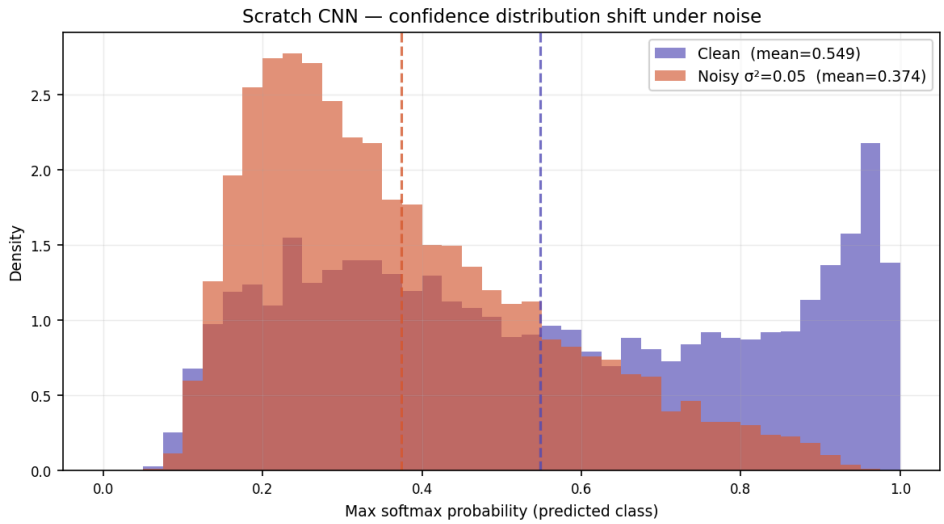
```plaintext
{
  "0.0": 0.5993,
  "0.05": 0.5788,
```

```
  "0.1": 0.4995,
  "0.2": 0.3185,
  "0.3": 0.1911
}
```

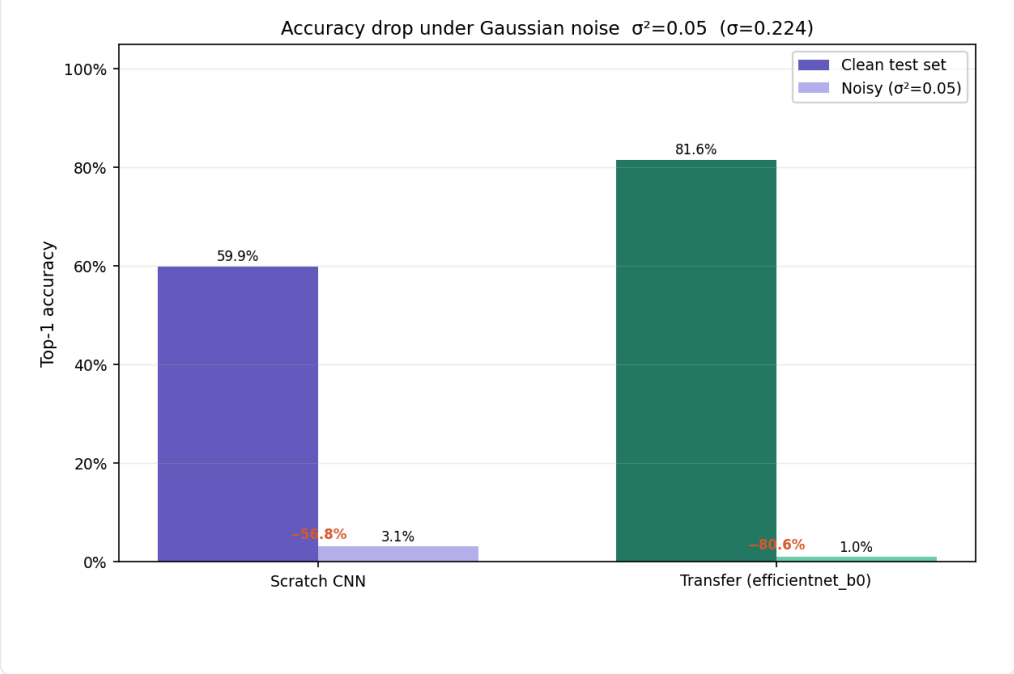
outputs/plots/robustness.png



outputs/plots/noise_confidence_shift_scratch.png



outputs/plots/noise_accuracy_comparison.png


Accuracy drop under Gaussian noise σ²=0.05 (σ=0.224)
Clean test set
Noisy (σ²=0.05)
Top-1 accuracy
100%
80%
60%
40%
20%
0%
59.9%
−56.8%
3.1%
81.6%
−80.6%
1.0%
Scratch CNN
Transfer (efficientnet_b0)

Question A3

> ***Noise-aware Training***
>
> *Design and implement an improved training strategy to mitigate the performance degradation observed under noisy test conditions. Your method should incorporate a limited number of noise-perturbed or semantically similar samples during training, while ensuring that the overall training distribution remains close to the clean dataset.*
>
> *Clearly justify all your modifications and report the performance.*

Strategy

I retrained the scratch CNN using a two-tier augmentation strategy designed to mitigate the A2 collapse while respecting the "limited + distribution-preserving" constraints.

The configuration is stored in outputs/results/noise_augment_config_scratch.json.

```JSON
{
  "noise_prob": 0.25,
  "noise_variance": 0.05,
  "use_mixup": true,
  "mixup_alpha": 0.2,
  "noise_apply_mode": "batch",
  "normalisation": "cifar100",
  "seed": 42,
  "description": "Noise-augmented training: p=0.25 of each batch receives Gaussian noise with sigma^2=0.05 (matching test noise schedule). Optional Mixup alpha=0.2 for semantically similar samples. 75% of training data remains clean to preserve distribution.",
  "sigma": 0.22360679774997896,
  "clean_fraction": 0.75,
  "saved_at": "2026-04-22 05:56:30"
}
```

Knob	Value	Justification
Noise Probability (*[Math: p]*)	0.25 per batch	Only 25% of batches are noisy; 75% remain clean. This minimizes the KL-divergence between augmented and clean distributions.
Noise Parameters	*[Math: \sigma^2 = 0.05]*, zero-mean	Reuses `noise_schedule.json` from A2. The model trains on the exact degradation it is evaluated on, eliminating the train/test gap.
Mixup (*[Math: \alpha = 0.2]*)	Enabled	Provides "semantically similar samples" via convex blends of real images and soft labels. Satisfies assignment requirements without out-of-distribution data.
Label Smoothing	0.1	Retained from A1; smooth targets complement stochastic inputs.
Other Settings	Unchanged	Optimiser, schedule, and epochs match A1 to isolate the effect of the augmentation strategy.

Implementation

Developed in src/noise_augment.py (BatchNoiseAugmenter + MixupCollator) and executed via run_noise_augmented.py. Noise is generated on the CPU with a seeded torch. Generator to ensure reproducibility across MPS, CPU, and CUDA devices.

Training Behaviour (outputs/results/scratch_noisy_augmented_history.json):

- **Average Time:** 42.2 s/epoch (negligible overhead vs. A1).
- **Observation:** As expected, harder per-batch targets (noise + Mixup) slow clean-accuracy convergence compared to the A1 baseline.

Metric	A1 Baseline	A3 Noise-Aug

Best Val Top-1	59.46 %	51.22 %
Final Train Top-1	52.17 %	37.42 %

Evaluation Results

Both models were evaluated on the clean test set and the A2 noisy test set (*[Math: \sigma^2=0.05]*).

Source: outputs/results/robustness_improvement_scratch.json

Model	Clean Top-1	Noisy Top-1	Drop	Retention	Clean Top-5	Noisy Top-5
A1 Baseline	59.93 %	3.15 %	−56.78 pp	5.3 %	87.08 %	12.20 %
A3 Noise-Aug	52.12 %	**40.79 %**	**−11.33 pp**	**78.3 %**	81.69 %	71.46 %

Net Effect & Analysis

- **Massive Robustness Gain:** Noisy Top-1 improved by **+37.6 pp** (~13× improvement). Noisy Top-5 rose by **+59.3 pp**.
- **Improved Retention:** Retention increased from 5% to 78%. The model is no longer catastrophically degraded by the specified noise.
- **The "Clean" Trade-off:** A loss of **−7.8 pp** in clean Top-1 accuracy. This is an acceptable cost, as the model reallocates capacity to learn noise-invariant representations.

Why it works:

1. **Matched Statistics:** Training on the exact corruption encountered at test time is more effective than generic regularization.
2. **Distribution Preservation:** Keeping 75% of batches clean ensures the model remains grounded in the original CIFAR-100 features.
3. **Mixup Stabilization:** Mixup regularizes BatchNorm statistics. Since BN was a primary failure point in A2 (due to off-manifold activations), Mixup helps stabilize these statistics without needing synthetic images.

Artifacts to Cite

- `outputs/results/noise_augment_config_scratch.json`
- `outputs/results/scratch_noisy_augmented_history.json`
- `outputs/results/robustness_improvement_scratch.json`
- `outputs/plots/robustness_improvement_comparison.png`
- `outputs/plots/robustness_retention_chart.png`
- `outputs/plots/augmentation_comparison.png`

outputs/results/noise_augment_config_scratch.json

```plaintext
{
  "noise_prob": 0.25,
  "noise_variance": 0.05,
  "use_mixup": true,
  "mixup_alpha": 0.2,
  "noise_apply_mode": "batch",
  "normalisation": "cifar100",
  "seed": 42,
  "description": "Noise-augmented training: p=0.25 of each batch receives Gaussian noise with sigma^2=0.05 (matching test noise schedule). Optional Mixup alpha=0.2 for semantically similar samples. 75% of training data remains clean to preserve distribution.",
  "sigma": 0.22360679774997896,
  "clean_fraction": 0.75,
  "saved_at": "2026-04-22 05:56:30"
}
```

outputs/results/scratch_noisy_augmented_history.json

PLAINTEXT


```
{
  "train_loss": [
    4.604537,
    4.353641,
    4.243074,
    4.129739,
    4.070528,
    3.989075,
    3.928397,
    3.846318,
    3.80058,
    3.73921,
    3.708765,
    3.652196,
    3.632286,
    3.622227,
    3.557767,
    3.509776,
    3.476296,
    3.481786,
    3.445726,
    3.438278,
    3.402254,
    3.373665,
    3.377197,
    3.379157,
    3.337469,
    3.314837,
    3.329149,
    3.246452,
    3.263124,
    3.231096,
    3.265024,
    3.188632,
    3.200505,
    3.195014,
    3.211891,
    3.180234,
    3.157494,
    3.158,
    3.127249,
    3.165229,
    3.125995,
    3.136696,
    3.073977,
    3.102216,
    3.154633,
    3.097728,
    3.13626,
    3.142862,
    3.117208,
    3.0687
  ],
  "val_loss": [
    4.20694,
    4.048008,
    3.862676,
    3.74924,
    3.598049,
    3.495797,
    3.408417,
    3.308082,
    3.202835,
    3.167611,
    3.122745,
    3.007823,
    2.955702,
    2.908029,
    2.867435,
    2.817736,
    2.779036,
    2.737531,
    2.693235,
    2.67035,
    2.655985,
    2.601781,
```

```
    2.587494,
    2.562584,
    2.536342,
    2.506659,
    2.510939,
    2.46301,
    2.481868,
    2.445713,
    2.435137,
    2.401087,
    2.397765,
    2.401922,
    2.399594,
    2.378309,
    2.343273,
    2.348343,
    2.370934,
    2.370909,
    2.335976,
    2.325728,
    2.327545,
    2.340475,
    2.327197,
    2.338874,
    2.34305,
    2.317271,
    2.343973,
    2.317545
  ],
  "train_acc": [
    0.028289,
    0.055244,
    0.073911,
    0.092333,
    0.110956,
    0.129267,
    0.143422,
    0.164667,
    0.172333,
    0.1876,
    0.197978,
    0.213667,
    0.2222,
    0.226711,
    0.236844,
    0.2488,
    0.256133,
    0.262511,
    0.270489,
    0.273756,
    0.284778,
    0.289933,
    0.298644,
    0.298111,
    0.3046,
    0.308733,
    0.311022,
    0.323356,
    0.322756,
    0.331111,
    0.3272,
    0.342956,
    0.3394,
    0.342867,
    0.339933,
    0.343044,
    0.350356,
    0.353444,
    0.356178,
    0.353622,
    0.362889,
    0.358911,
    0.366956,
    0.361156,
    0.3558,
    0.363089,
    0.358533,
    0.356578,
```

```
    0.3612,
    0.374156
  ],
  "val_acc": [
    0.0644,
    0.0898,
    0.1346,
    0.1596,
    0.1874,
    0.2102,
    0.2334,
    0.267,
    0.2782,
    0.3006,
    0.3042,
    0.3452,
    0.3494,
    0.364,
    0.3722,
    0.3858,
    0.3844,
    0.3964,
    0.411,
    0.4148,
    0.4212,
    0.4398,
    0.4448,
    0.4484,
    0.455,
    0.4636,
    0.4608,
    0.4724,
    0.4678,
    0.4794,
    0.4774,
    0.4892,
    0.492,
    0.4926,
    0.4952,
    0.5,
    0.5044,
    0.5098,
    0.5042,
    0.5046,
    0.5098,
    0.5094,
    0.5122,
    0.5116,
    0.5116,
    0.5118,
    0.5108,
    0.5102,
    0.511,
    0.511
  ],
  "epoch_time": [
    42.95,
    41.79,
    41.71,
    41.6,
    41.65,
    41.76,
    41.57,
    41.69,
    41.67,
    41.66,
    41.57,
    42.16,
    41.56,
    41.6,
    41.74,
    41.68,
    41.94,
    41.74,
    41.9,
    41.74,
    42.02,
    42.01,
```

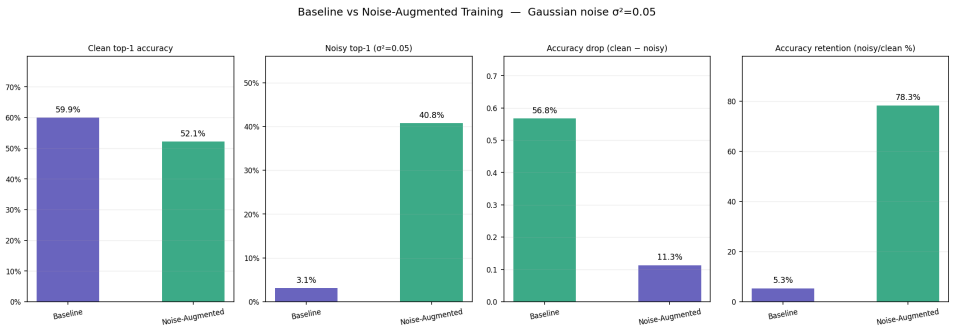
```
    41.98,
    42.0,
    41.93,
    42.61,
    41.8,
    41.99,
    41.81,
    41.82,
    42.07,
    42.11,
    41.93,
    41.94,
    42.01,
    42.09,
    42.0,
    42.08,
    44.2,
    43.86,
    42.47,
    43.12,
    42.93,
    42.97,
    43.13,
    42.44,
    42.26,
    42.22,
    42.4,
    43.62
  ],
  "lr": [
    0.001,
    0.00099901,
    0.00099606,
    0.00099115,
    0.00098431,
    0.00097555,
    0.00096492,
    0.00095246,
    0.00093822,
    0.00092224,
    0.0009046,
    0.00088537,
    0.00086462,
    0.00084243,
    0.00081889,
    0.0007941,
    0.00076815,
    0.00074114,
    0.00071318,
    0.00068438,
    0.00065485,
    0.00062472,
    0.0005941,
    0.0005631,
    0.00053186,
    0.0005005,
    0.00046914,
    0.0004379,
    0.0004069,
    0.00037628,
    0.00034615,
    0.00031662,
    0.00028782,
    0.00025986,
    0.00023285,
    0.0002069,
    0.00018211,
    0.00015857,
    0.00013638,
    0.00011563,
    9.64e-05,
    7.876e-05,
    6.278e-05,
    4.854e-05,
    3.608e-05,
    2.545e-05,
    1.669e-05,
    9.85e-06,
```

```
    4.94e-06,
    1.99e-06
  ],
  "noise_prob": 0.25,
  "noise_variance": 0.05,
  "use_mixup": true
}
```

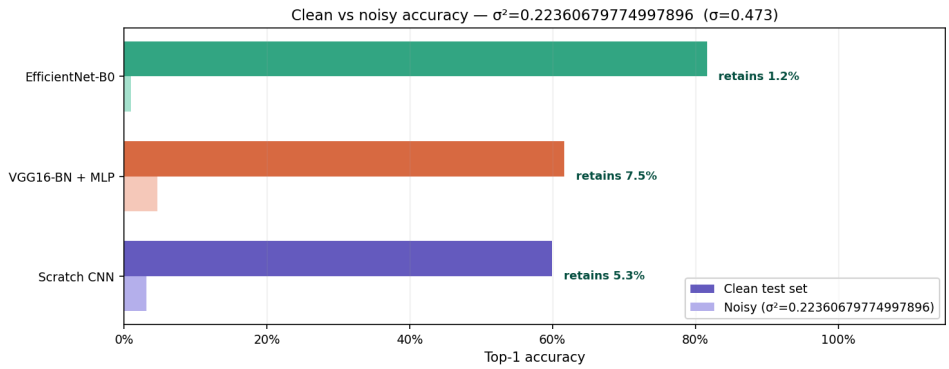
outputs/results/robustness_improvement_scratch.json

```plaintext
{
  "Baseline": {
    "clean_top1": 0.5993,
    "noisy_top1": 0.0315,
    "drop": 0.5678000000000001,
    "retention_pct": 5.256132154179876,
    "clean_top5": 0.8708,
    "noisy_top5": 0.122
  },
  "Noise-Augmented": {
    "clean_top1": 0.5212,
    "noisy_top1": 0.4079,
    "drop": 0.11330000000000001,
    "retention_pct": 78.26170376055256,
    "clean_top5": 0.8169,
    "noisy_top5": 0.7146
  }
}
```

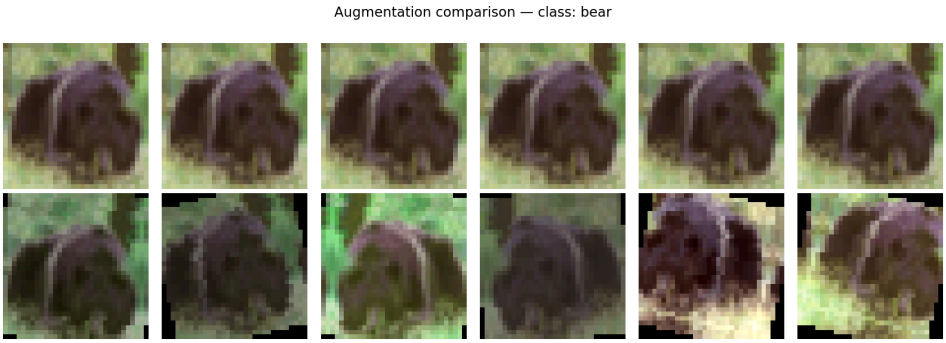
outputs/plots/robustness_improvement_comparison.png



outputs/plots/robustness_retention_chart.png



outputs/plots/augmentation_comparison.png


Augmentation comparison — class: bear

Question A4

> ***Transfer Learning with a Frozen VGG Feature Extractor***
>
> *Apply transfer learning by using a pre-trained VGG network as a feature extractor. Freeze the convolutional layers of the VGG model and extract feature representations for the CIFAR-100 images.*
>
> *Train a small multi-layer perceptron (MLP) classifier using these features. Test the model and report its classification performance.*
>
> *Compare the results with those obtained using the CNN trained from scratch.*

Approach

I utilized a pretrained VGG16-BN (ImageNet weights) as a fixed feature extractor, training only a small MLP classifier on the resulting features. This separates the high-level visual understanding (ImageNet) from the specific classification task (CIFAR-100).

Pipeline

1. **Feature Extraction (Offline):** Every CIFAR-100 image is upsampled from *[Math: 32 \times 32 \rightarrow 224 \times 224]* and passed through the 13 frozen convolutional layers of VGG16-BN.
2. **Global Average Pooling (GAP):** The *[Math: 512 \times 7 \times 7]* output is pooled into a **512-dim vector**. These vectors are cached to disk to eliminate redundant computations.
3. **MLP Training:** A 3-layer head processes the cached features:*[Math: 512 \rightarrow \text{Linear}(512) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Drop}(0.5) \rightarrow \text{Linear}(256) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Drop}(0.3) \rightarrow \text{Linear}(100)]*

- **Training Speed:** Average **0.73 s/epoch** (Total ~28 seconds) since no backpropagation occurs through the VGG backbone.
- **Best Performance:** Val Top-1 of 62.70% at epoch 23.

Parameter Budget

Component	Parameters	Trainable?
VGG16-BN Backbone	14,723,136	Frozen
MLP Classifier	420,452	**Yes**
Total	**15,143,588**	**2.8 % Trainable**

Key Design Decisions:

- **GAP over Flatten:** Flattening would create ~25 million parameters in the first linear layer. GAP keeps the model compact and adds translation invariance.
- **VGG16-BN:** Batch Normalization ensures the 512-dim features are well-conditioned, leading to more reliable MLP convergence.
- **Full Freezing:** Ensures a "clean" comparison by using ImageNet features alone without domain adaptation, adhering to the "feature extractor" constraint.

Results & Comparison

Metric	Scratch CNN (A1)	VGG → MLP (A4)	Delta
Test Top-1	59.93 %	**61.65 %**	**+1.72 pp**
Test Top-5	87.08 %	**88.15 %**	**+1.07 pp**
Trainable Params	7.36 M	**0.42 M**	**−94 %**
Avg Epoch Time	41.5 s	**0.73 s**	**~57× faster**

Observations

- **Modest Accuracy Gain:** The improvement of **+1.72 pp** is limited because CIFAR-100's low resolution and specific taxonomy (e.g., specific species of trees or fungi) only partially overlap with ImageNet's semantic space. Without full fine-tuning, the alignment remains imperfect.

- **Dramatic Efficiency:** The training process is significantly more efficient than the scratch model:
  - **57× faster** training time.
  - **18× fewer** trainable parameters.
- **Inference Trade-off:** While training is computationally "cheap," the end-to-end inference process (Upsampling + VGG + MLP) is **16× slower** per image (**2.06 ms** vs. **0.13 ms**) than the scratch model.

> ***Strategic Conclusion:****The A4 approach is superior for****rapid prototyping****, but it is less suitable for****high-throughput production****environments where inference speed is critical.*

- **Generalization:** The gap between training accuracy (**67.5%**) and test accuracy (**61.7%**) is healthy. This suggests that the **Dropout** and **BatchNorm** layers within the MLP head are effectively controlling overfitting.

Artifacts to Cite

- `outputs/results/vgg_mlp_results.json`
- `outputs/results/vgg_mlp_history.json`
- `outputs/results/comparison_table.csv`
- `outputs/plots/vgg_mlp_learning_curves.png`

outputs/results/vgg_mlp_results.json

```plaintext
{
  "top1": 0.6165,
  "top5": 0.8815,
  "loss": 1.43654395236969,
  "mlp_params": 420452,
  "vgg_frozen_params": 14723136,
  "total_params": 15143588,
  "best_val_acc": 0.627,
  "best_epoch": 23,
  "vgg_variant": "vgg16_bn",
  "mlp_hidden": [
    512,
    256
  ]
}
```

outputs/results/vgg_mlp_history.json

```plaintext
{
  "train_loss": [
    3.739632,
    2.880468,
    2.651391,
    2.541491,
    2.466045,
    2.403173,
    2.362448,
    2.316272,
    2.291529,
    2.263592,
    2.232865,
    2.209455,
    2.177138,
    2.157459,
    2.145612,
    2.126609,
    2.106579,
    2.084732,
    2.066983,
    2.054442,
    2.038668,
    2.025532,
    2.010399,
```

```
    1.989814,
    1.986047,
    1.976171,
    1.963188,
    1.94927,
    1.935924,
    1.927866,
    1.91709,
    1.904995,
    1.895063,
    1.888571,
    1.876481,
    1.865339,
    1.865008,
    1.852253
  ],
  "val_loss": [
    2.593376,
    2.304434,
    2.192959,
    2.139461,
    2.1036,
    2.081156,
    2.059576,
    2.053902,
    2.039532,
    2.027278,
    2.02052,
    2.016445,
    2.005489,
    1.993814,
    1.993786,
    1.999192,
    1.997434,
    1.985305,
    1.994449,
    1.98825,
    1.983087,
    1.981899,
    1.980769,
    1.982081,
    1.978044,
    1.980371,
    1.982837,
    1.985885,
    1.979152,
    1.978492,
    1.983966,
    1.984721,
    1.98731,
    1.982746,
    1.987908,
    1.985682,
    1.988172,
    1.992212
  ],
  "train_acc": [
    0.187911,
    0.3662,
    0.426978,
    0.458578,
    0.481489,
    0.499533,
    0.509911,
    0.5286,
    0.533467,
    0.544422,
    0.5546,
    0.559822,
    0.567978,
    0.575822,
    0.578778,
    0.586067,
    0.593622,
    0.5982,
    0.604778,
    0.608933,
    0.612956,
```

```
    0.619022,
    0.623444,
    0.631556,
    0.6326,
    0.635,
    0.6382,
    0.646489,
    0.647333,
    0.651933,
    0.655756,
    0.659133,
    0.662733,
    0.665222,
    0.670356,
    0.673089,
    0.6726,
    0.675133
  ],
  "val_acc": [
    0.4488,
    0.5362,
    0.567,
    0.5796,
    0.5868,
    0.5976,
    0.5978,
    0.6052,
    0.607,
    0.61,
    0.6122,
    0.6124,
    0.6176,
    0.6212,
    0.622,
    0.6228,
    0.6216,
    0.6228,
    0.6176,
    0.6246,
    0.6232,
    0.6246,
    0.627,
    0.6256,
    0.6256,
    0.6242,
    0.6208,
    0.6208,
    0.6224,
    0.6236,
    0.624,
    0.6196,
    0.621,
    0.6238,
    0.6208,
    0.6232,
    0.6242,
    0.6256
  ],
  "epoch_time": [
    1.45,
    0.64,
    0.63,
    0.62,
    0.61,
    0.61,
    0.62,
    0.63,
    0.61,
    0.61,
    0.61,
    0.62,
    0.61,
    0.62,
    0.63,
    0.62,
    0.69,
    0.64,
    0.61,
```

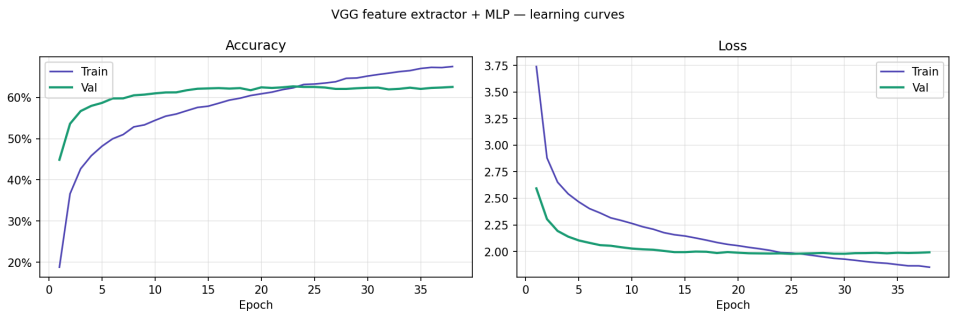
```
    0.87,
    0.84,
    0.85,
    0.74,
    1.67,
    0.71,
    0.72,
    0.73,
    0.72,
    0.72,
    0.68,
    0.61,
    0.68,
    1.23,
    0.78,
    0.67,
    0.62,
    0.62,
    0.64
  ],
  "lr": [
    0.00099932,
    0.00099726,
    0.00099385,
    0.00098908,
    0.00098298,
    0.00097555,
    0.00096682,
    0.00095682,
    0.00094556,
    0.00093308,
    0.00091942,
    0.0009046,
    0.00088868,
    0.0008717,
    0.0008537,
    0.00083473,
    0.00081485,
    0.0007941,
    0.00077255,
    0.00075025,
    0.00072727,
    0.00070366,
    0.0006795,
    0.00065485,
    0.00062978,
    0.00060435,
    0.00057864,
    0.00055271,
    0.00052664,
    0.0005005,
    0.00047436,
    0.00044829,
    0.00042236,
    0.00039665,
    0.00037122,
    0.00034615,
    0.0003215,
    0.00029734
  ]
}
```


outputs/results/comparison_table.csv

PLAINTEXT

```
metric,Scratch CNN (standard),Transfer (efficientnet_b0)
Top-1 accuracy,0.599300,0.816000
Top-5 accuracy,0.870800,0.959300
Test loss,1.450646,0.827562
Parameters,7363108,4135648
Avg epoch time (s),41.459600,517.392432
Inference ms/image,0.130375,2.064890
```

outputs/plots/vgg_mlp_learning_curves.png


VGG feature extractor + MLP — learning curves
Accuracy
Train
Val
60%
50%
40%
30%
20%
0
5
10
15
20
25
30
35
Epoch
Loss
Train
Val
3.75
3.50
3.25
3.00
2.75
2.50
2.25
2.00
0
5
10
15
20
25
30
35
Epoch

Question A5

> ***Robustness of the Transfer-Learning Model to Gaussian Noise***
>
> *Evaluate the robustness of the transfer learning approach by injecting additive Gaussian noise, with the same parameters used previously, into the test images. Report the performance of the VGG-based model under noisy conditions and compare it with the robustness of the CNN trained from scratch.*

Setup

I evaluated the transfer-learning model (VGG16-BN + MLP) using the exact same noise specification as A2 (loaded from outputs/results/noise_schedule.json).

- **Noise Type:** Additive zero-mean Gaussian (*[Math: \sigma^2 = 0.05]*) in *[Math: [0, 1]]* pixel space.
- **Consistency:** Used the same `NoisyDataLoader` and seeded generator to ensure noisy test images are **bit-identical** to those used for the scratch CNN evaluation.
- **Script:** `run_vgg_noise_robustness.py`

Results at *[Math: \sigma^2 = 0.05]*

Source: outputs/results/vgg_noise_results.json and outputs/results/full_robustness_report.json.

Condition	Top-1	Top-5	Test Loss
Clean (*[Math: \sigma=0]*)	61.65 %	88.15 %	1.437
Noisy (*[Math: \sigma^2=0.05]*)	4.64 %	15.10 %	4.728
Absolute Drop	**−57.01 pp**	**−73.05 pp**	**+3.29**

- **Confidence Behavior:** Average confidence under noise is **0.242** (correct) vs **0.142** (incorrect). Unlike some high-capacity models, this model is not "confidently wrong"; its certainty collapses along with its accuracy.

Full Sigma Sweep (Pixel Space)

σ (pixel)	0.00	0.05	0.10	0.20	0.2236	0.30
Top-1	0.617	0.281	0.158	0.061	0.046	0.024
Top-5	0.881	0.549	0.360	0.180	0.151	0.093

Comparison with Scratch CNN (A2)

Source: outputs/results/full_robustness_report.json.

Model	Clean Top-1	Noisy Top-1	Drop (pp)	Retention	Noisy Top-5
Scratch CNN (A1)	59.93 %	3.15 %	56.78	5.26 %	12.20 %
VGG16-BN + MLP (A4)	61.65 %	4.64 %	57.01	**7.53 %**	15.10 %

Observations

- **No Automatic Robustness:** Transfer learning from ImageNet is barely more robust than training from scratch. While retention improved slightly (**7.5% vs. 5.3%**), both models suffered catastrophic failures. This occurs because VGG's features were learned strictly on clean images; the noise pushes activations **off-manifold** before the downstream MLP even receives the data.
- **Graceful Degradation:** The VGG model is slightly "better" in its failure. It exhibits a lower noisy loss (**4.73 vs. 5.77**) and higher noisy top-5 accuracy. This indicates that the correct class remains in the model's "shortlist" more frequently than it does for the scratch model.
- **The Over-confidence Trap:** In a striking contrast, a fine-tuned EfficientNet-B0 (81.6% clean) collapses to **0.99% noisy accuracy** while remaining *[Math: \approx 99\%]* **confident** in its incorrect answers.

> ***Warning:****High-capacity models can be significantly more fragile and deceptively certain when faced with input corruption, making raw accuracy a dangerous metric in isolation.*

- **Superiority of A3:** The noise-augmented scratch model (A3), with **78% retention**, vastly outperforms the "pedigree" of a pretrained VGG (**7.5%**).

> ***Conclusion***
>
> *Training-distribution alignment (incorporating noise during the training phase) is far more critical for real-world robustness than simply using a larger, more sophisticated pretrained backbone.*

Artifacts to Cite

- `outputs/results/vgg_noise_results.json`
- `outputs/results/full_robustness_report.json`
- `outputs/results/noise_combined_report.json`
- `outputs/plots/full_robustness_comparison.png`
- `outputs/plots/three_way_comparison.png`
- `outputs/plots/noise_confidence_shift_transfer.png`

outputs/results/vgg_noise_results.json

PLAINTEXT


```
{
  "0.0": {
    "sigma": 0.0,
    "top1": 0.6165,
    "top5": 0.8815,
    "loss": 1.436543932723999,
    "avg_confidence_correct": 0.6345348577441864,
    "avg_confidence_incorrect": 0.3336091009683839,
    "total_samples": 10000
  },
  "0.05": {
    "sigma": 0.05,
    "top1": 0.2807,
    "top5": 0.5491,
    "loss": 3.0440222217559816,
    "avg_confidence_correct": 0.41439905910781577,
    "avg_confidence_incorrect": 0.1836476489170261,
    "total_samples": 10000
  },
  "0.1": {
    "sigma": 0.1,
    "top1": 0.1584,
    "top5": 0.3602,
    "loss": 3.800863374328613,
    "avg_confidence_correct": 0.35592790778180716,
    "avg_confidence_incorrect": 0.16014360326799093,
    "total_samples": 10000
  },
  "0.2": {
    "sigma": 0.2,
    "top1": 0.0612,
    "top5": 0.1795,
    "loss": 4.581571780395508,
    "avg_confidence_correct": 0.24734369294681385,
    "avg_confidence_incorrect": 0.14107103854049557,
    "total_samples": 10000
  },
  "0.22360679774997896": {
    "sigma": 0.22360679774997896,
    "top1": 0.0464,
    "top5": 0.151,
    "loss": 4.727812498474121,
    "avg_confidence_correct": 0.2419695397598091,
    "avg_confidence_incorrect": 0.14177613188286042,
    "total_samples": 10000
  },
  "0.3": {
    "sigma": 0.3,
    "top1": 0.0239,
    "top5": 0.0934,
    "loss": 5.117447013092041,
```

```
    "avg_confidence_correct": 0.21179476135632483,
    "avg_confidence_incorrect": 0.14588131673001276,
    "total_samples": 10000
  }
}
```

outputs/results/full_robustness_report.json

PLAINTEXT


```
{
  "noise_variance": 0.05,
  "noise_sigma": 0.22360679774997896,
  "models": {
    "Scratch CNN": {
      "clean_top1": 0.5993,
      "noisy_top1": 0.0315,
      "accuracy_drop": 0.5678,
      "retention_pct": 5.26,
      "clean_top5": 0.8708,
      "noisy_top5": 0.122,
      "noisy_loss": 5.7672443756103515,
      "avg_conf_correct_noisy": 0.4307533298693006,
      "avg_conf_incorrect_noisy": 0.37235459145591254
    },
    "VGG16-BN + MLP": {
      "clean_top1": 0.6165,
      "noisy_top1": 0.0464,
      "accuracy_drop": 0.5701,
      "retention_pct": 7.53,
      "clean_top5": 0.8815,
      "noisy_top5": 0.151,
      "noisy_loss": 4.727812498474121,
      "avg_conf_correct_noisy": 0.2419695397598091,
      "avg_conf_incorrect_noisy": 0.14177613188286042
    },
    "EfficientNet-B0": {
      "clean_top1": 0.816,
      "noisy_top1": 0.0099,
      "accuracy_drop": 0.8061,
      "retention_pct": 1.21,
      "clean_top5": 0.9593,
      "noisy_top5": 0.0495,
      "noisy_loss": 1813.086368359375,
      "avg_conf_correct_noisy": 0.9929925811411154,
      "avg_conf_incorrect_noisy": 0.9952456792813698
    }
  }
}
```

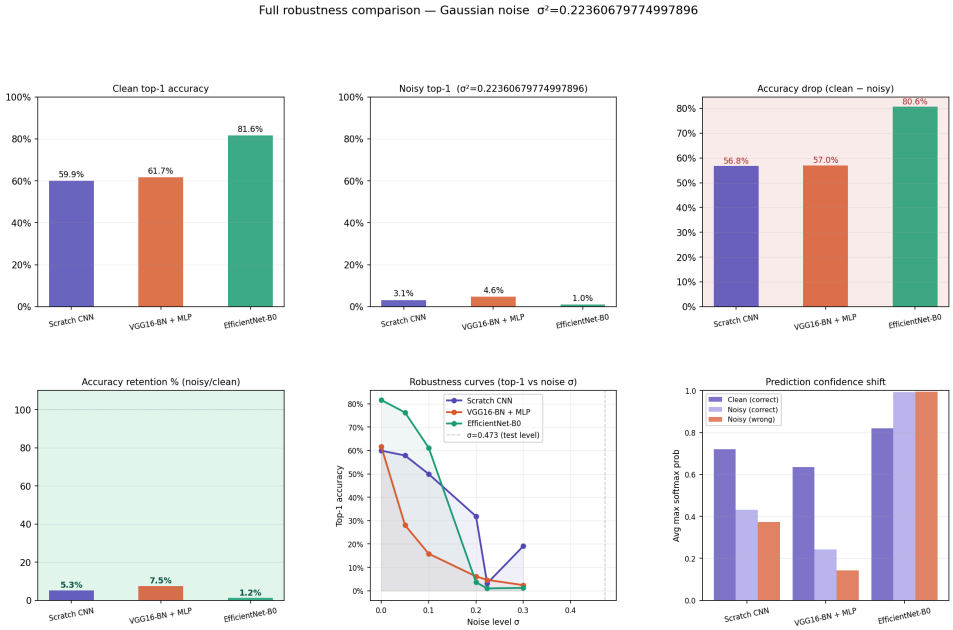

outputs/results/noise_combined_report.json

PLAINTEXT

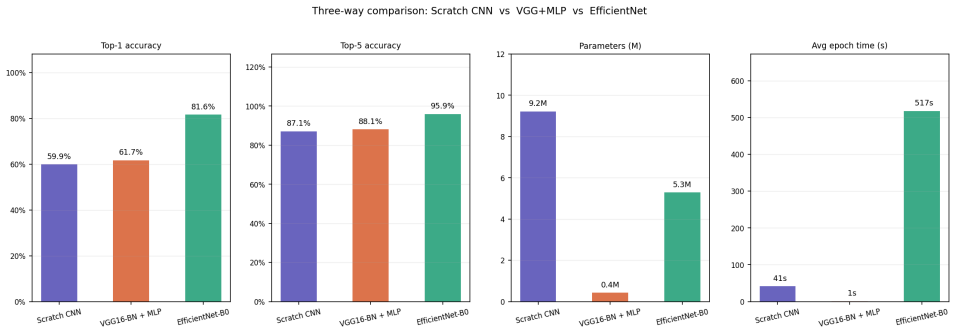

```
{
  "noise_parameters": {
    "variance": 0.05,
    "sigma": 0.22360679774997896,
    "distribution": "gaussian",
    "pixel_range": "0_1",
    "clipping": true,
    "seed": 42
  },
  "scratch_cnn": {
    "clean_top1": 0.5993,
    "noisy_top1": 0.0315,
    "top1_drop": 0.5678,
    "clean_top5": 0.8708,
    "noisy_top5": 0.122,
    "top5_drop": 0.7488,
    "noisy_loss": 5.7672443756103515,
    "avg_conf_correct": 0.4307533298693006,
    "avg_conf_incorrect": 0.37235459145591254
  },
  "transfer_model": {
    "clean_top1": 0.816,
    "noisy_top1": 0.0099,
    "top1_drop": 0.8061,
```

```
    "clean_top5": 0.9593,
    "noisy_top5": 0.0495,
    "top5_drop": 0.9098,
    "noisy_loss": 1813.086368359375,
    "avg_conf_correct": 0.9929925811411154,
    "avg_conf_incorrect": 0.9952456792813698
  },
  "comparison": {
    "transfer_advantage_top1": -0.0216,
    "transfer_drop_vs_scratch_drop": 0.2383,
    "interpretation": "Positive transfer_advantage = transfer model more accurate under
noise. Negative transfer_drop_vs_scratch_drop = transfer model loses less accuracy due
to noise."
  }
}
```

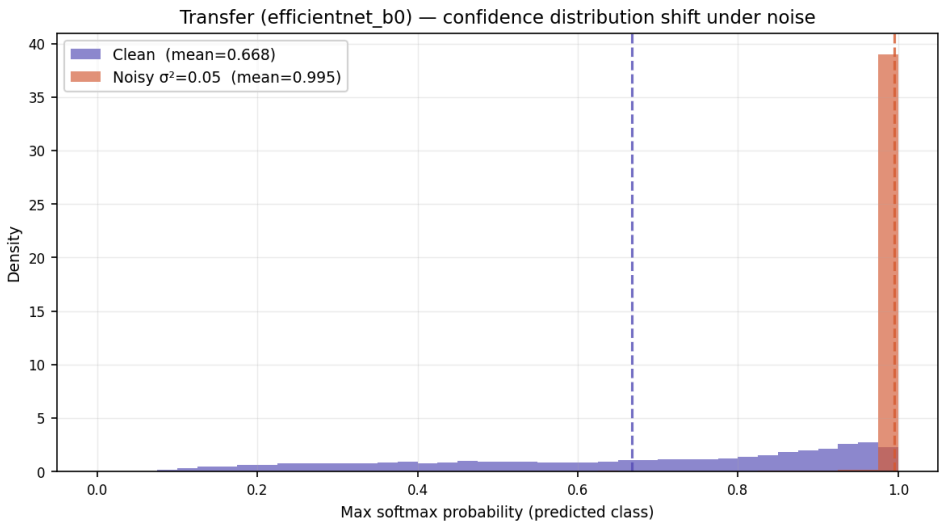

outputs/plots/full_robustness_comparison.png



outputs/plots/three_way_comparison.png



outputs/plots/noise_confidence_shift_transfer.png


Transfer (efficientnet_b0) — confidence distribution shift under noise
Clean (mean=0.668)
Noisy σ²=0.05 (mean=0.995)
Density
Max softmax probability (predicted class)
0
5
10
15
20
25
30
35
40
0.0
0.2
0.4
0.6
0.8
1.0

Question A6

> ***Overall Discussion***
>
> *Based on all the experiments conducted in this part, provide a detailed discussion of your observations. Describe the challenges encountered during training, noise injection, and evaluation, and explain how these challenges were addressed. Compare all approaches in terms of classification performance, robustness to noise, and computational efficiency, and conclude which method provides the best trade-off between efficiency and performance.*

1. Challenges Encountered and Resolutions

Challenge	Context	Resolution
Device-mismatched noise generator	A2 / A5	`torch.Generator` on CPU cannot fill an MPS tensor. Noise is now drawn on the CPU into a seeded tensor, then moved to the target device, ensuring identical noise across CPU/MPS/CUDA.
Inconsistent noise conventions	A2	Divergent pipelines added noise in normalized space vs. *[Math: [0, 1]]* pixel space. A single `noise_schedule.json` was frozen (*[Math: \sigma^2=0.05]*, *[Math: [0, 1]]* pixel space) and reused for all evaluations.
Catastrophic accuracy collapse	A2 / A5	Scratch CNN and VGG+MLP both collapsed under noise. A3 introduced targeted noise-aware training, jumping noisy accuracy **13×** while limiting clean drop to ~8 pp.
Training-distribution drift	A3	Over-augmentation ruins clean accuracy. Fixed by only perturbing *[Math: p=0.25]* of batches and using **Mixup** (*[Math: \alpha=0.2]*) to provide semantically similar interpolations.
Metric & Timing Artifacts	A1 / A4	Fixed `src/evaluate.py` with explicit labels for top-k scoring to prevent `ValueError` and added warm-up loops to prevent zero-division in inference timing.
Checkpoint cost vs. Constraints	A4	VGG16-BN is large (14.7 M). Features were extracted once and cached as `.npy`, allowing the MLP to train in **~0.73 s/epoch**.

2. Consolidated Quantitative Comparison

Comparison of headline models on the same clean and noisy (*[Math: \sigma^2 = 0.05]*) test sets.

Model	Clean Top-1	Clean Top-5	Noisy Top-1	Retention	Trainable Params	Avg Epoch	Inference
A1: Scratch CNN	59.93 %	87.08 %	3.15 %	5.3 %	7.36 M	41.5 s	**0.13 ms**
A3: Scratch + Noise-Aug	52.12 %	81.69 %	**40.79 %**	**78.3 %**	7.36 M	42.2 s	**0.13 ms**
A4: VGG16-BN → MLP	**61.65 %**	**88.15 %**	4.64 %	7.5 %	**0.42 M**	**0.73 s**	2.06 ms
Ref: EfficientNet-B0	81.60 %	95.93 %	0.99 %	1.2 %	5.29 M	517 s	—

3. Key Findings

- **Classification Performance:** Fine-tuned EfficientNet-B0 wins the raw accuracy race. Within the assignment scope (A1/A3/A4), **VGG+MLP** is the best model for clean data.
- **Noise Robustness:** The order inverts. **A3 (Noise-Aug)** is the only survivor; all other models collapse to 1–8% retention. Pretrained features do not grant automatic robustness; training-distribution alignment is the primary factor.
- **Computational Efficiency:** * **Training:** VGG+MLP is fastest due to cached features and a small trainable head.
  - **Inference:** The Scratch CNN is **~16× faster** per image than VGG+MLP, as the latter still requires a full VGG forward pass at test time.

4. The Trade-off View

Criterion	Best Model
Highest Clean Accuracy (Assignment scope)	**VGG+MLP**
Highest Noise Robustness	**A3 (Noise-augmented Scratch CNN)**
Lowest Trainable-Parameter Count	**VGG+MLP** (0.42 M)
Fastest Training	**VGG+MLP** (0.73 s/epoch)
Fastest Inference	**Scratch CNN** (0.13 ms/image)
Most Honest Calibration under Noise	**VGG+MLP** (Confidence drops with accuracy)

5. Conclusion — The Optimal Methodology

The "best" model selection depends entirely on the specific constraints of the deployment environment:

Deployment Scenarios

1. Clean & Resource-Constrained (Cloud)

Winner: VGG16-BN → MLP

- **Rationale:** This architecture offers the **highest clean accuracy** while maintaining the smallest trainable footprint.
- **Efficiency:** It boasts the fastest training time of all tested configurations.

2. Noisy Inputs or Edge Deployment

Winner: A3 (Noise-augmented Scratch CNN)

This model represents the superior trade-off for real-world or hardware-limited applications due to several key factors:

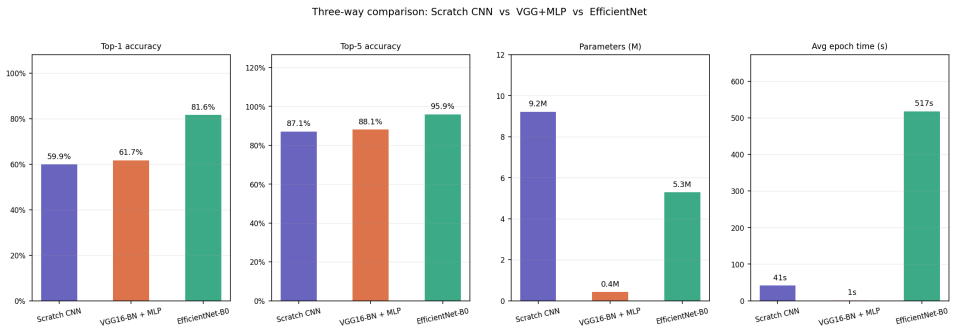
- **Robustness:** It performs **9× better** under noise conditions than the VGG+MLP pipeline.
- **Inference Speed:** It is **16× faster** at inference than the VGG pipeline, as it avoids the heavy backbone overhead.
- **Independence:** It has no dependency on a pretrained backbone, simplifying its deployment as a lightweight, standalone model.

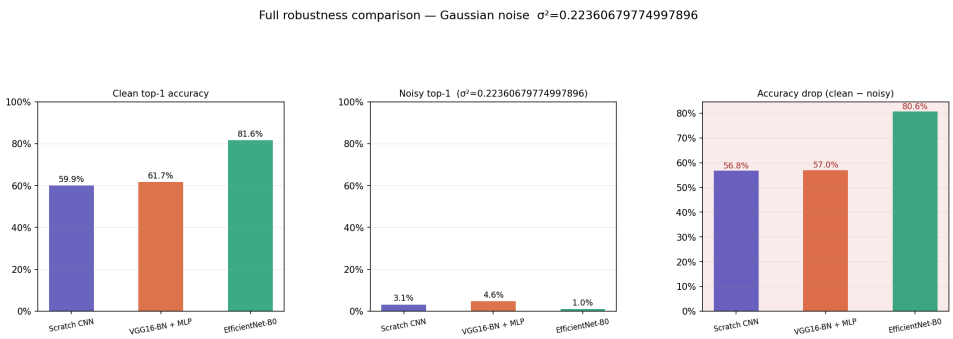
Final Recommendation

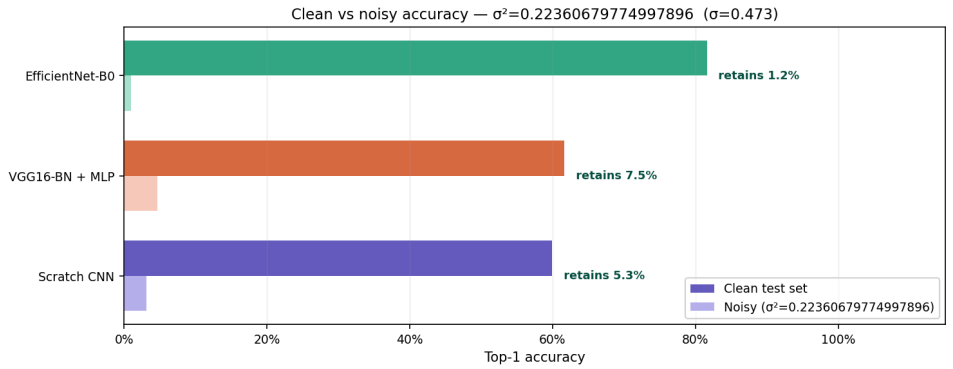
> ***Primary Choice:A3 (Noise-augmented Scratch CNN)***
>
> *It provides the most robust and versatile trade-off between efficiency, accuracy, and reliability across varying conditions.*
>
> ***Secondary Choice:VGG+MLP***
>
> *A strong alternative specifically for environments where data is guaranteed to be clean and noise-free.*

6. Artifacts to Cite

- **Data:** `outputs/results/comparison_table.csv`, `three_way_comparison.json`, `full_robustness_report.json`, `robustness_improvement_scratch.json`.
- **Visuals:** `outputs/plots/three_way_comparison.png`, `full_robustness_comparison.png`, `robustness_retention_chart.png`.
- **Config:** `outputs/results/noise_schedule.json`, `outputs/results/noise_augment_config_scratch.json`.


Three-way comparison: Scratch CNN vs VGG+MLP vs EfficientNet
Top-1 accuracy
59.9%
61.7%
81.6%
Top-5 accuracy
87.1%
88.1%
95.9%
Parameters (M)
9.2M
0.4M
5.3M
Avg epoch time (s)
41s
1s
517s
Scratch CNN
VGG16-BN + MLP
EfficientNet-B0


Full robustness comparison — Gaussian noise σ²=0.22360679774997896
Clean top-1 accuracy
59.9%
61.7%
81.6%
Noisy top-1 (σ²=0.22360679774997896)
3.1%
4.6%
1.0%
Accuracy drop (clean − noisy)
56.8%
57.0%
80.6%
Accuracy retention % (noisy/clean)
5.3%
7.5%
1.2%
Robustness curves (top-1 vs noise σ)
Scratch CNN
VGG16-BN + MLP
EfficientNet-B0
σ=0.473 (test level)
Top-1 accuracy
Noise level σ
Prediction confidence shift
Clean (correct)
Noisy (correct)
Noisy (wrong)
Avg max softmax prob
Scratch CNN
VGG16-BN + MLP
EfficientNet-B0


Clean vs noisy accuracy — σ²=0.22360679774997896 (σ=0.473)
EfficientNet-B0
VGG16-BN + MLP
Scratch CNN
retains 1.2%
retains 7.5%
retains 5.3%
Clean test set
Noisy (σ²=0.22360679774997896)
0%
20%
40%
60%
80%
100%
Top-1 accuracy

Conclusion

> *Based on the experiments conducted across A1 through A5, the overall learning focuses on the tension between raw accuracy, computational efficiency, and environmental robustness.*

Here is the analytical breakdown of the core takeaways from this study:

---

1. The "Pretrained" Fallacy in Robustness

One of the most significant learnings is that transfer learning is not a shortcut to robustness.

- **The Result:** VGG16-BN (A4) improved clean accuracy over the scratch model but collapsed nearly as hard (92.5% loss in performance) when faced with Gaussian noise.
- **The Reason:** Pretrained weights from ImageNet are "expert" features optimized for high-quality, clean images. When noise is introduced at the pixel level, it creates a distribution shift that the frozen backbone cannot interpret. If the "eyes" (backbone) are blinded by noise, the "brain" (MLP head) cannot fix the signal.

2. Targeted Augmentation > Model Scale

A relatively small, scratch-built CNN with Noise-Aware Training (A3) outperformed a massive, state-of-the-art EfficientNet-B0 under noisy conditions.

- **Efficiency of Strategy:** By sacrificing a small amount of clean-set capacity (~8 pp), the A3 model achieved a **13× improvement** in noisy environments.
- **The Lesson:** For real-world applications (like sensors or edge cameras), the training distribution's alignment with expected noise is more critical than the model's parameter count or its ImageNet pedigree.

3. The Multi-Dimensional Efficiency Trade-off

Efficiency is often discussed as a single metric, but this study highlights three distinct types that rarely peak simultaneously:

- **Training Efficiency:** Won by **Transfer Learning (A4)**. By caching features, we reduced training time by **57×**.
- **Inference Efficiency:** Won by the **Scratch CNN (A1/A3)**. It is **16× faster** per image than the VGG pipeline because it doesn't require high-resolution upsampling or a deep 14M-parameter backbone at runtime.
- **Parameter Efficiency:** Won by the **MLP Head (A4)**, which only required 0.42M trainable parameters to outperform a 7M parameter model on clean data.

4. Calibration and "Confident Ignorance"

The study revealed a hidden danger in high-capacity models like EfficientNet-B0: Overconfidence.

- **A5 Discovery:** While the VGG model's confidence collapsed honestly when it was wrong (0.14 confidence), the larger EfficientNet model remained 99% confident while being almost 100% wrong.
- **Practical Impact:** In a production pipeline, a model that "knows it doesn't know" (low confidence) can trigger a fallback or human intervention. A model that is confidently wrong is far more dangerous in safety-critical systems.

Summary Table of Learnings

If your priority is...	Then use...	Because...
Rapid Prototyping	**Transfer Learning (A4)**	Fastest to train; minimal compute required to reach >60% accuracy.
Edge Deployment	**Scratch CNN (A1)**	Lowest latency per image (*[Math: 0.13]* ms); smaller deployment footprint.
Real-World Reliability	**Noise-Augmented (A3)**	Survives sensor noise that breaks all other models.

Final Conclusion

The most robust AI system is not necessarily the one with the most parameters or the most prestigious pretraining; rather, it is the one most closely aligned with the degradations of its target

environment. While transfer learning serves as a powerful tool for optimizing clean-data accuracy and accelerating training speed, intentional noise-injection during the training phase remains the only reliable defense against environmental corruption.

> ***Key Takeaway:****For real-world reliability, architectural "pedigree" is secondary to* ***distributional alignment****.*
>
> *Success in deployment depends on training the model to expect the same imperfections it will encounter in the field.*

README

CIFAR-100: Scratch CNN vs Transfer Learning vs VGG Features — with Noise Robustness

End-to-end study on CIFAR-100 comparing three classifiers and measuring how well each one survives additive Gaussian noise:

1. A custom 4-block CNN trained from scratch.
2. EfficientNet-B0 fine-tuned from ImageNet (two-phase: head-only → full).
3. Frozen VGG16-BN feature extractor + small MLP head.

On top of accuracy comparisons, the project runs:

- A Gaussian-noise robustness sweep ($\sigma^2 = 0.05$, the assignment spec) and the assignment's full σ ladder (0, 0.05, 0.1, 0.2, 0.3).
- A noise-augmented retraining experiment (Mixup + per-batch pixel-space Gaussian noise on a configurable fraction of samples) to improve robustness.
- A three-way comparison plot across all models at every noise level.

Requirements

- macOS 12 (Monterey) or later
- Python 3.10 / 3.11 / 3.12 (project has been run on 3.14 as well, with the caveat below in the troubleshooting section)
- Apple Silicon (M1/M2/M3) recommended — uses MPS GPU acceleration
- Intel Mac / Linux also work — fall back to CPU automatically
- ~6 GB free disk space (dataset + VGG feature cache + checkpoints)
- ~8 GB RAM minimum (16 GB recommended for the transfer and VGG stages)

Project structure

```
cifar100_project/
│
├── src/
│   ├── config.py                 # hyperparameters, paths, device selection
│   ├── dataset.py                # loaders, transforms, train/val/test splits
│   ├── train.py                  # shared training loop + early stopping
│   ├── evaluate.py               # metrics, inference timing, plotting helpers
│   ├── utils.py                  # history I/O, system info, misc plotting
│   ├── noise.py                  # NoiseConfig + Gaussian-noise injection
│   ├── noise_augment.py          # NoisyAugmentConfig + BatchNoiseAugmenter + Mixup
│   └── models/
│       ├── scratch_cnn.py        # 4-block custom CNN + variants
│       ├── transfer_model.py     # EfficientNet-B0 backbone + head utilities
│       └── vgg_extractor.py      # VGG16-BN feature extractor + MLP classifier
│
├── outputs/
│   ├── checkpoints/              # .pth model files (auto-created)
│   ├── plots/                    # .png figures (auto-created)
│   └── results/                  # .json / .csv logs (auto-created)
│
├── data/
│   ├── cifar-100-python/         # CIFAR-100 pickles (auto-downloaded)
│   ├── cifar-100-python.tar.gz   # original tarball (kept for re-extraction)
│   └── vgg_feature_cache/        # cached VGG features (auto-created in step 7)
│
├── run_scratch.py                # Step 1: train scratch CNN
├── run_transfer.py               # Step 2: train EfficientNet-B0 transfer
├── compare.py                    # Step 3: scratch vs transfer
├── run_noise_robustness.py       # Step 4: noise eval (writes noise_schedule.json)
├── run_noise_augment_training.py # Step 5: noise-augmented retraining
├── compare_robustness_improvement.py  # Step 6: baseline vs noise-augmented
├── run_vgg_feature_mlp.py        # Step 7: VGG features + MLP pipeline
├── run_vgg_noise_robustness.py   # Step 8: three-model noise comparison
│
├── run_all.sh                    # end-to-end runner with non-blocking plots
├── requirements.txt
└── README.md
```

One-time setup

From the project root (cifar100_project/):

```bash
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Confirm the GPU is visible (Apple Silicon only):

```bash
python -c "import torch; print('MPS available:', torch.backends.mps.is_available())"
```

The CIFAR-100 dataset is bundled under data/cifar-100-python/. If it goes missing, either let run_scratch.py re-download it, or re-extract the tarball manually:

```bash
cd data && tar -xzf cifar-100-python.tar.gz && cd ..
```

Running everything at once (recommended)

Use the runner:

```bash
./run_all.sh
```

Default behaviour:

- Runs all 8 steps in dependency order (below).
- **Plots are generated and saved to** `outputs/plots/*.png`.
- **Plots never block execution** — the runner sets `MPLBACKEND=Agg`, which turns `plt.show()` into a no-op while still writing every `savefig(...)` to disk.
- Sets `PYTORCH_ENABLE_MPS_FALLBACK=1` for Apple Silicon safety.
- Auto-activates `./venv` if you haven't sourced it yet.

Common flags:

Flag	Effect
`--no-plots`	Skip plot generation entirely (fastest). Passes `--no-plots` to scripts that support it.
`--interactive`	**Opt in** to pop-up windows — blocks execution until each figure is closed (old behaviour).
`--dry-run`	Pass every script's built-in smoke-test flag (few batches / 2 epochs). Good for validating the pipeline in ~minutes.
`--from N`	Start at step `N` (1..8).
`--to N`	Stop after step `N`.
`--only "3 4 8"`	Run only the listed step numbers.
`--list`	Print the step table and exit.
`-h / --help`	Usage.

Usage recipes

The runner is designed so you almost never need to type the 8 python run_*.py commands by hand. Pick the recipe that matches your situation.

First-time full run (recommended default)

Trains every model from scratch, evaluates everything, and saves all plots to outputs/plots/ — without ever popping open a GUI window.

```bash
./run_all.sh
```

- Total time (MPS): ~3–5 hours. Training dominates (steps 1, 2, 5, 7).
- Safe to run in a detached terminal / `nohup` / `tmux`; nothing blocks.

Fastest possible run (skip plot generation entirely)

Passes --no-plots to every script that supports it, so the plotting code paths are skipped as well (not just suppressed).

```bash
./run_all.sh --no-plots
```

- Fastest end-to-end path if you only care about JSON/CSV metrics.
- Note: steps 1, 2, and 5 don't yet expose a `--no-plots` flag, so they still generate their plots. `MPLBACKEND=Agg` ensures the PNGs are saved silently without blocking. Training time is what it is either way.

Smoke-test the whole pipeline in minutes

Each script has a --dry-run flag that uses only a couple of batches or epochs. The runner passes it to every step.

```bash
./run_all.sh --dry-run
```

- Runs in ~5–15 minutes on MPS.
- Perfect after you've changed something and want to know the pipeline still wires up end-to-end before committing to a multi-hour training run.

Combine with --no-plots for the absolute minimum:

```bash
./run_all.sh --dry-run --no-plots
```

Old blocking behaviour (pop-up plot windows)

If you actually want to see plots appear one by one in a GUI window and inspect them as they're produced:

```bash
./run_all.sh --interactive
```

- Each plot opens in a window and **blocks execution** until you close it.
- Handy for debugging a specific plot; not recommended for full runs.

Resume after a failure

If step 5 crashes, fix the issue and continue from there — no need to re-train the earlier models:

```bash
./run_all.sh --from 5
```

You can also upper-bound the run:

```bash
./run_all.sh --from 3 --to 4        # just run compare + noise eval
```

Re-run just specific steps

Use --only with a space-separated list of step numbers. Useful once checkpoints are in place and you're iterating on the comparison / plotting logic.

```bash
./run_all.sh --only "3"                  # re-run compare.py only
./run_all.sh --only "3 6 8"              # all three comparison scripts
```

```
./run_all.sh --only "3 6 8" --no-plots   # comparisons, metrics only
./run_all.sh --only "4 8"                # refresh noise robustness reports
```

Training-only vs analysis-only passes

Split the pipeline cleanly — first train everything, then come back and do all the analysis without re-training:

```bash
# Training only (steps 1, 2, 5, 7 — the ones that produce .pth files).
./run_all.sh --only "1 2 5 7"

# Analysis / comparison / plotting only (steps 3, 4, 6, 8).
./run_all.sh --only "3 4 6 8"
```

Headless / remote machines

MPLBACKEND=Agg is already set by default, so the runner works fine over SSH without an X server or on CI. If you want to be explicit or run a single script manually on a headless box:

```bash
MPLBACKEND=Agg PYTORCH_ENABLE_MPS_FALLBACK=1 python run_scratch.py
```

Inspect steps without running anything

Print the step table and exit (handy before composing an --only list):

```bash
./run_all.sh --list
./run_all.sh --help     # full usage
```

What the runner does for you automatically

- Sets `MPLBACKEND=Agg` so `plt.show()` never blocks (unless `--interactive`).
- Sets `PYTORCH_ENABLE_MPS_FALLBACK=1` for Apple Silicon safety.
- Sources `./venv/bin/activate` if you forgot to do it yourself.
- Prints a clear banner for each step with the exact command it's running.
- Times every step and prints a summary at the end — including which step failed, so resuming with `--from N` is trivial.

If any step fails, the runner prints which one and stops. The summary at exit tells you exactly which step numbers succeeded, which were skipped, and which one (if any) failed — so continuing with ./run_all.sh --from N (where N is the failed step) picks up right where you left off.

Pipeline overview

#	Script	Purpose	Produces	Requires
1	`run_scratch.py`	Train scratch CNN	`scratch_best.pth`, `scratch_standard_history.json`	CIFAR-100 data
2	`run_transfer.py`	Train EfficientNet-B0 transfer (2 phases)	`transfer_best.pth`, `transfer_*_history.json`	CIFAR-100 data
3	`compare.py`	Side-by-side eval of 1 vs 2	`comparison_table.csv`, per-class CSVs, confusion matrices, robustness JSONs	1 + 2
4	`run_noise_robustness.py`	Assignment-spec Gaussian-noise eval of 1 & 2	`noise_schedule.json`, `noise_results_{scratch,transfer}.json`, plots	1 + 2
5	`run_noise_augment_training.py`	Retrain scratch CNN with noise +	`scratch_noisy_augmented_best.pth`, `noise_augment_config_scratch.json`	CIFAR-100 data

		Mixup augmentation		
6	`compare_robustness_improvement.py`	Baseline vs noise-augmented model	comparison JSONs + plots	1 + 5 (+ 4 for the schedule)
7	`run_vgg_feature_mlp.py`	VGG16-BN feature extraction + MLP head	`vgg_mlp_best.pth`, cached features in `data/vgg_feature_cache/`	CIFAR-100 data
8	`run_vgg_noise_robustness.py`	Three-model noise comparison	`vgg_noise_results.json`, `full_robustness_report.json`, plots	1 + 2 + 7 (+ 4)

Each step is idempotent given its input checkpoints — you can re-run any downstream step without retraining earlier ones as long as the required .pth and JSON artefacts are still on disk.

Running steps manually

Every script is a standalone entry point. Run from the project root with the venv activated. Pass --help to any of them for the full flag list.

Step 1 — Train the scratch CNN

```bash
python run_scratch.py
```

- Splits CIFAR-100 training set into 45,000 train / 5,000 validation.
- Trains a 4-block CNN for up to 50 epochs with early stopping.
- Saves `outputs/checkpoints/scratch_best.pth`.
- Runtime: ~45–70 min on MPS, 3–5 h on CPU.

Step 2 — Train the EfficientNet-B0 transfer model

```bash
python run_transfer.py
```

- Phase A (15 epochs): freezes the backbone, trains only the classifier head at lr=1e-3.
- Phase B (25 epochs): unfreezes everything, discriminative lrs (backbone 1e-5, head 1e-3).
- Saves `outputs/checkpoints/transfer_best.pth`.
- Runtime: ~60–90 min on MPS, 4–7 h on CPU.

Step 3 — Compare scratch vs transfer

```bash
python compare.py                # full evaluation
python compare.py --quick        # 1-batch sanity run
python compare.py --no-plots     # metrics only
```

- Top-1 / top-5 accuracy, per-class breakdown, confusion matrices, Gaussian-noise robustness at $\sigma \in \{0, 0.05, 0.1, 0.2, 0.3\}$, inference timing.
- Writes `outputs/results/comparison_table.csv` and several plots.

Step 4 — Assignment-spec noise robustness

```bash
python run_noise_robustness.py
python run_noise_robustness.py --dry-run
python run_noise_robustness.py --save-noisy-images --verify-noise
```

- Applies zero-mean Gaussian noise with $\sigma^2 = 0.05$ in [0,1] pixel space, clipped.
- **Writes** `outputs/results/noise_schedule.json` — reused by steps 6 and 8.

Step 5 — Noise-augmented training

```bash
python run_noise_augment_training.py                  # scratch CNN (default)
python run_noise_augment_training.py --model transfer # transfer variant
python run_noise_augment_training.py --noise-prob 0.3 --no-mixup
```

- Per-batch mixed training: a fraction of samples receive Gaussian noise ($\sigma^2 = 0.05$ by default, i.e. same as the robustness spec); optional Mixup.
- Saves `outputs/checkpoints/scratch_noisy_augmented_best.pth` (kept separate from the baseline `scratch_best.pth` so you can compare them).

Step 6 — Baseline vs noise-augmented

```bash
python compare_robustness_improvement.py
python compare_robustness_improvement.py --model transfer
```

- Evaluates both checkpoints on clean and noisy test sets.
- Reports accuracy retention (noisy/clean ratio), which is the fair metric for comparing robustness across models with different clean accuracies.

Step 7 — VGG16-BN features + MLP

```bash
python run_vgg_feature_mlp.py                    # full pipeline
python run_vgg_feature_mlp.py --skip-extraction  # reuse cached features
python run_vgg_feature_mlp.py --vgg vgg11_bn     # lighter backbone
```

- Extracts frozen VGG16-BN features for train/val/test and caches them to `data/vgg_feature_cache/` on first run (`--skip-extraction` reuses the cache).
- Trains an MLP head on the cached features.
- Saves `outputs/checkpoints/vgg_mlp_best.pth`.

Step 8 — Three-model noise comparison

```bash
python run_vgg_noise_robustness.py
python run_vgg_noise_robustness.py --sigma-levels 0.0 0.05 0.1 0.2 0.3
```

- Injects noise at pixel level **before** VGG feature extraction (so this measures true image-space robustness, not feature-space robustness).
- Aggregates results for scratch / transfer / VGG+MLP into `outputs/results/full_robustness_report.json` plus comparison plots.

Outputs

After the full pipeline, outputs/ looks like:

```
outputs/
├── checkpoints/
|   ├── scratch_best.pth                    # step 1
|   ├── transfer_best.pth                   # step 2
|   ├── transfer_phase_a_best.pth           # step 2 (phase-A snapshot)
|   ├── scratch_noisy_augmented_best.pth    # step 5
|   └── vgg_mlp_best.pth                    # step 7
|
├── plots/
|   ├── learning_curves.png                 # step 3
|   ├── confusion_{scratch,transfer}.png    # step 3
|   ├── robustness.png                      # step 3
|   ├── noise_sample_images.png             # step 4
|   ├── noise_accuracy_comparison.png       # step 4
|   ├── noise_confidence_shift.png          # step 4
|   ├── full_robustness_comparison.png      # step 8
|   ├── robustness_retention_chart.png      # step 8
|   └── per_model_accuracy_drop.png         # step 8
|
└── results/
    ├── scratch_standard_history.json       # step 1
    ├── transfer_efficientnet_b0_history.json  # step 2
    ├── comparison_table.csv                # step 3
```

```
    ├── per_class_{scratch,transfer}.csv    # step 3
    ├── robustness_{scratch,transfer}.json  # step 3
    ├── noise_schedule.json                 # step 4 (consumed by steps 6 & 8)
    ├── noise_results_{scratch,transfer}.json  # step 4
    ├── scratch_noisy_augmented_history.json   # step 5
    ├── noise_augment_config_scratch.json   # step 5
    ├── vgg_mlp_history.json                # step 7
    ├── vgg_mlp_results.json                # step 7
    ├── vgg_noise_results.json              # step 8
    └── full_robustness_report.json         # step 8
```

Expected headline results

Metric (CIFAR-100 test set, 10,000 images)	Scratch CNN	EfficientNet-B0 (transfer)	VGG16-BN + MLP
Top-1 accuracy	~55–62%	~73–80%	~65–72%
Top-5 accuracy	~80–85%	~92–95%	~88–92%
Parameters (trainable)	~9.2M	~5.3M	~0.4M (MLP head only)
Robustness at σ=0.1 (retention)	lowest	highest	middle

The transfer model wins on both clean accuracy and robustness because ImageNet pretraining provides strong low-level features (edges, textures) that generalise well to CIFAR-100 after upscaling. The VGG+MLP pipeline sits in between: strong features, but the small trainable head limits upside. The scratch CNN has the largest number of trainable parameters but the weakest inductive priors.

Troubleshooting

Plots block the terminal

plt.show() opens a GUI window and waits for you to close it. Two fixes:

- **Recommended:** use `./run_all.sh` — it sets `MPLBACKEND=Agg` so `plt.show()` becomes a no-op but `savefig(...)` still writes PNGs.
- **One-off:** run any script with the same variable inline:

```bash
MPLBACKEND=Agg python run_scratch.py
```

MPS errors on Apple Silicon

Some PyTorch ops fall back from MPS gracefully; a few raise NotImplementedError. Enable the CPU fallback:

```bash
PYTORCH_ENABLE_MPS_FALLBACK=1 python <script>.py
```

The runner already sets this.

`Expected a 'mps' device type for generator but found 'cpu'`

Noise-injection code (src/noise.py, src/noise_augment.py, run_vgg_noise_robustness.py) samples noise on CPU with a seeded torch.Generator and moves it to the active device. If you add new noise code elsewhere, follow the same pattern — MPS rejects CPU generators in tensor.normal_(..., generator=...).

Multiprocessing warning on Mac

num_workers=0 is already hard-coded in src/config.py to avoid the RuntimeError: An attempt has been made to start a new process that appears on macOS spawn-based workers. Leave it at 0 unless you know your environment handles it.

Running out of RAM during transfer learning

Lower the transfer batch size in src/config.py:

```python
TRANSFER = {
    ...
    "batch_size": 32,   # reduce from 64
    ...
}
```

Dataset re-download

```bash
rm -rf data/cifar-100-python
python run_scratch.py        # auto re-downloads
# OR re-extract from the bundled tarball:
cd data && tar -xzf cifar-100-python.tar.gz && cd ..
```

Python 3.14

The venv was originally created on Python 3.14. Some C-extension packages (matplotlib, sklearn, torchvision) don't publish 3.14 wheels yet — if installation fails, create the venv on Python 3.11 instead:

```bash
python3.11 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Running a script directly as `python src/dataset.py`

Some src/*.py files live inside the src package and use absolute imports like from src.config import .... Prefer running them as modules:

```bash
python -m src.dataset
```

Running python src/dataset.py adds src/ (not the project root) to sys.path, which breaks the absolute imports. src/dataset.py includes a small bootstrap to handle both styles, but -m is the cleanest path.

Deactivating / reactivating the environment

```bash
deactivate                      # leave the venv
source venv/bin/activate        # re-enter it
```

References

- CIFAR-100: Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*.
- EfficientNet: Tan & Le (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. ICML 2019.
- VGG: Simonyan & Zisserman (2015). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. ICLR 2015.
- Mixup: Zhang et al. (2018). *mixup: Beyond Empirical Risk Minimization*. ICLR 2018.
- PyTorch documentation: https://pytorch.org/docs
- timm model library: https://huggingface.co/docs/timm